Golf Simulator in Scilab: Capturing a swing and estimating the cary with a tiny low-cost Radar

Matthieu PHILIPPE
Publication Perso

November 8, 2026

Contents

1 Introduction 2

2 Golf Ball Flight 4
2.1 Drag Force . . . 5
2.1.1 Air characteristics . . . 5
2.1.2 Ball's characteristics in the air . . . 5
2.2 Magnus Effect . . . 6
2.3 Gravity Force . . . 8
2.4 Launch Velocity . . . 8
2.5 Balls Velocity with Miss sweet spot . . . 9
2.6 Club Face Initial Velocity . . . 9
2.7 Complete Model of flight . . . 10
2.7.1 Complete Model of Flight (Matrix/Vector Form) . . . 10
2.7.1.1 Initial Conditions . . . 11
2.7.2 Complete Model of Flight (Tensor Notation) . . . 11
2.7.2.1 Governing Equation . . . 11
2.7.2.2 Drag Force . . . 11
2.7.2.3 Magnus Force . . . 11
2.7.2.4 Gravity Force . . . 11
2.7.2.5 Initial Conditions . . . 12
2.8 External Function for the Ordinary Differential Equation solver . . . 12
2.8.1 Architecture and Design . . . 12
2.8.2 Equation Definition Functions . . . 12
2.8.3 Event Detection . . . 13
2.8.4 Numerical Integration . . . 14
2.8.5 Limitations and Potential Evolutions . . . 14

3 Bouncing and Rolling 16
3.1 Bouncing and Rolling . . . 16
3.1.1 Ball's Characteristics During Bouncing . . . 16
3.1.1.1 Impact Angle and Restitution . . . 16
3.1.1.2 Bounce Angle and Velocity Update . . . 17
3.1.1.3 Friction and Spin During Bounce . . . 17
3.1.2 Ball's Characteristics During Rolling . . . 18
3.1.2.1 Rolling Resistance . . . 19
3.1.2.2 Stopping Condition . . . 19
3.1.3 Limitations and Potential Evolutions . . . 20
3.2 Scilab Code Explanation for Golf Ball Simulation . . . 21
3.2.1 The `flightg` Function . . . 21
3.2.1.1 Inputs and Outputs . . . 21
3.2.1.2 Internal Logic . . . 22
3.2.2 The `rolling` Function . . . 23
3.2.2.1 Inputs and Outputs . . . 24

This document and the information contained herein are the property of Matthieu PHILIPPE.
They must not be copied or disclosed to third parties without prior written authorization.

3.2.2.2 Internal Logic . . . 24
3.2.3 The AltiSol and dAltiSol Functions . . . 25
3.2.3.1 The AltiSol Function . . . 25
3.2.3.2 The dAltiSol Function . . . 25
3.2.4 Event Detection Functions . . . 26
3.2.4.1 The events Function . . . 26
3.2.4.2 The events2 Function . . . 26
3.2.5 Numerical Integration with ode . . . 26
3.2.5.1 Calling the ode Function . . . 27

4 Hardware 28

5 Data Anlysis 31
5.1 Filtered Radar Captures . . . 31
5.1.1 Filtering . . . 32
5.1.2 Final Result . . . 33

6 Radar Doppler Processing Pipeline: from Scilab to Python 35
6.1 Two-radar geometry and the Doppler observables . . . 35
6.2 STFT and ridge tracking . . . 36
6.3 Speed: median of the post-impact ridge vs global max . . . 36
6.4 Spin: patch periodicity vs spectral excursion . . . 37
6.5 Stereo pairing of the two ridges . . . 38
6.6 Quality controls (“non identifiable” is a valid output) . . . 38
6.7 Scilab → Python: corrections and validation results . . . 38
6.8 Processing pipeline . . . 39
6.9 Spectrograms: measured captures vs theoretical model . . . 39

7 Listings 44
7.1 Golfball.sci . . . 44

List of Figures

1.1 HMI Of Simulation . . . 2
1.2 Radar Captures . . . 3

2.1 **Schéma des forces agissant sur une balle de golf en vol.** . . . 4
2.2 Velocity Components . . . 8
2.3 Club Path . . . 9

4.1 The suitecase . . . 28
4.2 Internal with Arduino, Radar, Amplifier, Audio Boards . . . 29
4.3 Tiny Arduino . . . 29
4.4 Audio Board . . . 29
4.5 Audio Amplifier Board . . . 30
4.6 Radar Board . . . 30

5.1 Wav Radar Captures and its echo . . . 31

This document and the information contained herein are the property of Matthieu PHILIPPE.
They must not be copied or disclosed to third parties without prior written authorization.

5.2 Captured Raw Data Wav . . . 32
5.3 FFT of Filtered Raw Data . . . 32
5.4 My power density Algorithm Results to identify Speed and BackSpin . . . 33
5.5 Results of normalized mean filtered power spectral density . . . 33
5.6 Results of the filtered Density and caracteristics identification . . . 34
5.7 Speed and Backspin echos . . . 34

6.1 Top view of the two-radar setup. f_C carries V_x (speed), f_Δ carries V_z (azimuth α in the x–z plane). The factor $2L/b$ corrects the parallax: the farther the ball, the smaller the frequency difference for the same V_z. . . . 36
6.2 The aluminium patch as a single dominant scatterer. Both the flash periodicity and the spectral excursion measure the *total* spin ω; neither separates backspin from sidespin. . . . 37
6.3 Python validation pipeline (`sci/pipe_validation.py`). Scilab equivalents: `animDensite` (STFT), `Info/GetSwing` (speed), `animHilbert` (envelope), `acos(f2/f1)` (rudimentary azimuth in `testsFFT.sci`). 39
6.4 Radar setup geometry (also in README). . . . 39
6.5 Patch on the ball (aluminium tape). . . . 40
6.6 Measured spectrogram, shot 123 m / 150 km/h / 5274 RPM (SkyTrak reference). . . . 40
6.7 Measured spectrogram, shot 131 m / 162 km/h / 6185 back + 7102 side RPM ($\beta \approx 48.9°$, strong curvature). . . . 41
6.8 Measured spectrogram, shot 137 m / 166 km/h / 4990+1312 RPM ($\beta \approx 14.7°$). . . . 41
6.9 Theoretical (generated) capture, config F7 151 km/h, launch 16.5°, spin 4000 RPM, side 700 RPM ($\beta = 10.1°$), club 115 km/h, carry 102 m. . . . 42
6.10 Generated capture, same config, swing-complete 450 ms version (v2). . . . 42
6.11 Generated capture with the real 137 m shot parameters (166 km/h, 4990/1312 RPM, club 118). 43
6.12 Model trajectory (flight + lateral deviation of the slice) for the 2025-11-08 generated capture. . 43

List of Tables

2.1 Table of my C_{l1} . . . 7

6.1 Measured results, 11 CSV (9 real F7 shots + 2 model signals). . . . 38

This document and the information contained herein are the property of Matthieu PHILIPPE.
They must not be copied or disclosed to third parties without prior written authorization.

Listings

3.1 Calling the ode function for the flight phase 27
7.1 Listing of the golf ball fligth 44

List of Equations

2.1 Flight Forces 4
2.2 Air Force Resistance 5
2.2 Ball Mass, Diameter, Rayon, Intertia, Section, Reyolds Number 5
2.5 Drag Coefficient, Drag Force 6
2.7 Magnus Lift effect 6
2.12 Magnus effect and different coefficient models 7
2.13 Angle between mouvment and spin axes 7
2.16 Gravity Force 8
2.21 Ball velocity and launch angle after the impact of the golf club 8
2.22 Intial ball velocity after missed the sweet spot 9

List of Remarks

1 Finally, the equation of the drag Force along the 3-axes becomes 2.6 6
2 Knowing 2.7 and 2.14 and 2.1, I chose the following Magnus force model 7
3 The Gravity Force 8

This document and the information contained herein are the property of Matthieu PHILIPPE.
They must not be copied or disclosed to third parties without prior written authorization.

List of definitions

1 The Air Drag Force 5
2 The Magnus Effect 6
3 The gravity force 8

ABSTRACT

By using the equations of golf ball flight dynamics, this article demonstrates how to code a swing simulator for a ball in Scilab.

The simulator uses an ODE (ordinary differential equation) solver from Scilab, the fluid dynamics equations on a ball (taking into account gravity, drag forces, and Magnus forces), the restitution of forces from a golf club applied to a ball for launches, its flight, bounce and roll.

In the second part, the article presents a method for capturing the speed of the ball in flight using 2 low-cost radars, coupled with an Arduino Mini and an adapted sound card connected to the wave radar's output .

The radar signal is recorded in WAV format and filtered to extract speed and spin informations during first meters of flight, using power spectral density in realtime mode.

Keywords: *Golf Ball, Simulator, Scilab, Radar, Spectral dentisty.*

This document and the information contained herein are the property of Matthieu PHILIPPE.
They must not be copied or disclosed to third parties without prior written authorization.

Chapter 1

Introduction

After spending a few years playing golf, I wanted to learn more about the dynamics of the interactions between the club, the ball, and the elements.

Of course, the academic literature and publications were quite abundant. But I came accross articles [PWB76] written in 1976 and [SS94] in 1994. The description of the ball's physics pushed me to undertake modeling this system in Scilab (fig 1.1). Subsequently, I did it in Java, C++, and then C# to use it under *Unity game engine*®™.

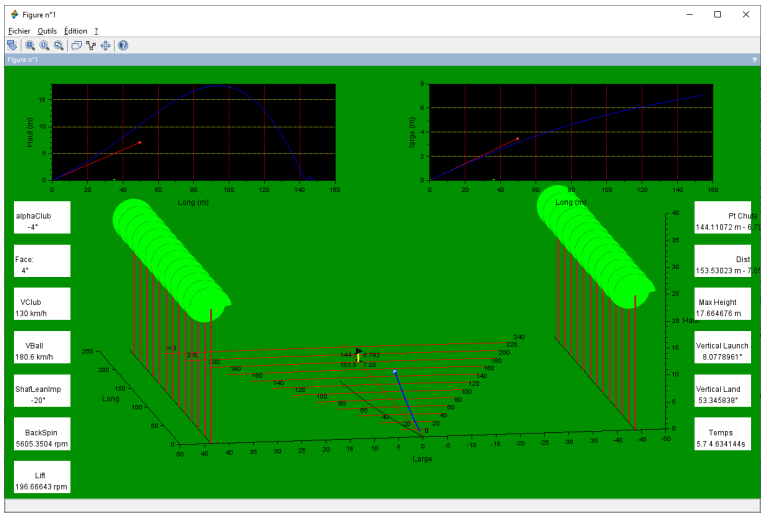


Figure 1.1: HMI Of Simulation

When the results were satisfactory, I thought about capturing the flight of a golf ball with instrumentation. After tests with sonar, which were largely insufficient, I took small presence detection radars for automatic doors.

I didn't know what I was getting into, but after a updating my knowledge on signal filtering and a few lines of programming in Scilab, I found a way to capture the essential data. Namely the speed and backspin of the flight of a golf ball and the club's echo since juste before smashing and during flight(figure 1.2).

This document and the information contained herein are the property of Matthieu PHILIPPE.
They must not be copied or disclosed to third parties without prior written authorization.

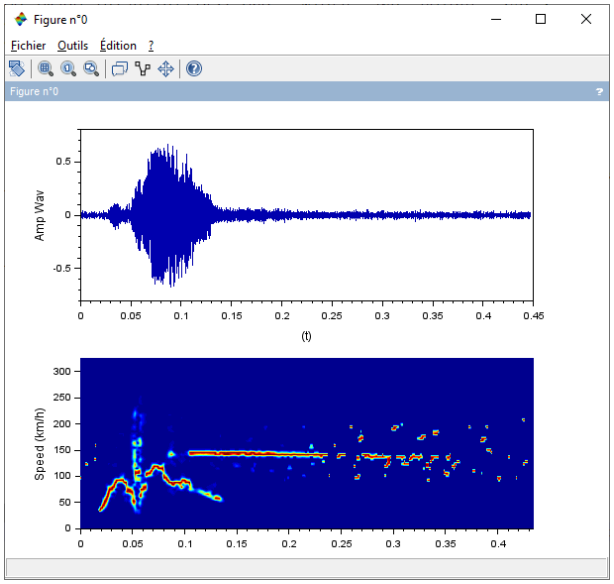


Figure 1.2: Radar Captures

This article describes all items I found, and how I set all them together. Equations, programs, filtering, estimators and electronics compounents to make a Virtual Golf Simulation.

All the equations presented in this article are derived from public publications or books, with references listed in the bibliography. I will not revisit the demonstrations and physical explanations, as they are not the focus of these pages. What matters to me is the relationship between this information and how I have used or adapted it.

This document and the information contained herein are the property of Matthieu PHILIPPE.
They must not be copied or disclosed to third parties without prior written authorization.

Chapter 2

Golf Ball Flight

The main equation is the ball's flight. The equation accounts 3 forces : Magnus Force, Air Drag Force and gravity, such as :

$$\sum \overrightarrow{F} = \overrightarrow{F_m} + \overrightarrow{F_d} + \overrightarrow{F_g} \tag{2.1}$$

—

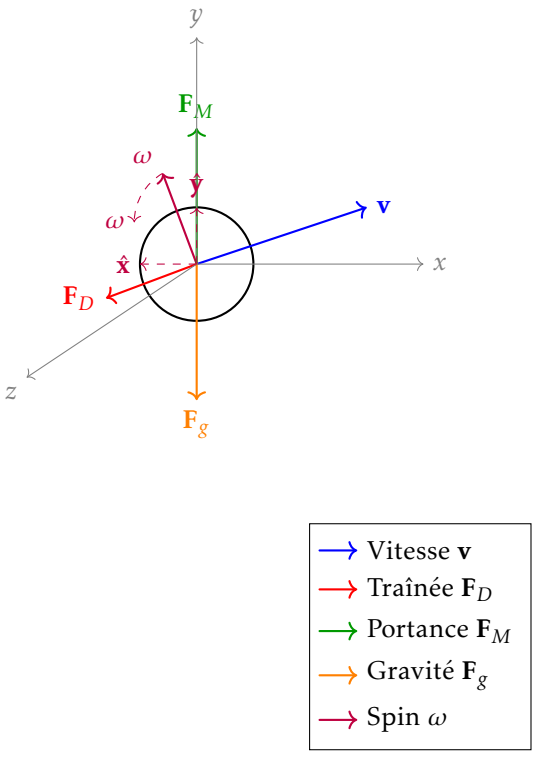


Figure 2.1: **Schéma des forces agissant sur une balle de golf en vol.**

—

This document and the information contained herein are the property of Matthieu PHILIPPE.
They must not be copied or disclosed to third parties without prior written authorization.

2.1 Drag Force

2.1.1 Air characteristics

Definition 1: The Air Drag Force	*The Air Drag Force is the resitance exposed on the cross-sectional area of an object, orthogonal to the displacement, determined by the relation given by [BR] as 2.2*

:

$$F_d(V) = -\frac{1}{2}.\rho(T).C_d.S.V^2(t) \tag{2.2}$$

where:

$$\begin{aligned}
\rho(T) &= 1,292 * \frac{273,15}{273,15+T} \\
\rho(T) &= Air\ density\ Rho\ depending\ on\ temperature\ T \\
T &= Temperature \\
C_d &= Drag\ Coefficient,\ depending\ on\ the\ object \\
V &= Object's\ Velocity \\
F_d(V) &= Drag\ Force\ due\ to\ Air\ Resistance\ depending\ on\ velocity\ V \\
S &= The\ cross-sectional\ area\ exposed,\ orthogonal\ to\ the\ displacement.
\end{aligned}$$

2.1.2 Ball's characteristics in the air

Considering those characteristics of a dimpled ball model,

$$\begin{aligned}
mass\ m &= 0,04593\ g \\
Dynamic\ viscosity\ \mu &= 1,5.10^{-6} m^2/s \\
Diameter\ D &= 0,04267\ m \\
Radius\ r &= \frac{D}{2} \\
Inertia\ I &= \frac{2}{5}.m.R^2 \\
Cross-section\ S &= \pi.R^2 \\
Ball\ Velocity &= V
\end{aligned}$$

We have an estimated models,

$$Re = \frac{\rho(T).V.D}{\mu} \tag{2.3}$$

$$C_d = 0,36 + \frac{24}{Re} + \frac{6}{1+\sqrt{Re}} \tag{2.4}$$

$$\overrightarrow{F_d} = -\frac{1}{2}.\rho(T).C_d.S.\sqrt{(V(x)^2+V(y)^2+V(z)^2)}.\widehat{V} \tag{2.5}$$

This document and the information contained herein are the property of Matthieu PHILIPPE.
They must not be copied or disclosed to third parties without prior written authorization.

where,

$$\begin{aligned} \widehat{V} &= Unit\ vector\ of\ the\ velocity \\ Re &= Reynolds\ Number\ estimator \\ C_d &= Drag\ Coefficient\ White's\ relation \\ \overrightarrow{F_d} &= Drag\ Force \end{aligned}$$

Remark 1: Finally, the equation of the drag Force along the 3-axes becomes 2.6	$$\overrightarrow{F_d} = \begin{pmatrix} -\frac{1}{2}.\rho(T).C_d.S.v_i^2 \\ -\frac{1}{2}.\rho(T).C_d.S.v_j^2 \\ -\frac{1}{2}.\rho(T).C_d.S.v_k^2 \end{pmatrix} \quad (2.6)$$

2.2 Magnus Effect

Definition 2: The Magnus Effect	*The Magnus [Mag53] effect is due to the spin of a cylindrical object, moving in a fluid (air). It is called the Robin effect for a ball.*

But by generalization, the Magnus effect name is applied to any type of object subjected to this suction force. This lift force occurs, altering the ballistic trajectory of the golf ball. The Force depends on the Lift coefficient (C_l), that is associated with the object, according to its shape, nature and texture. So that the force is written as a composant of Object Velocity, SpinRate and a coefficient of Magnus effect C_m (2.7),

$$\overrightarrow{F_m} = C_m.(\overrightarrow{\omega} \times \overrightarrow{V}) \quad (2.7)$$

where,

$$\begin{aligned} \omega &= spin\ in\ rad/s \\ V_p &= r\omega\ periferal\ velocity \\ V &= Ball\ Velocity \\ C_m &= Coefficient\ of\ Magnus\ effect \end{aligned}$$

The main bibliography for golf ball I used, is [PAL05] and [SS94]. I have found several ways to calculate the C_m, given in expressions bellow. The first simple aproximation is given by 2.8, and the Coefficient of Magnus effect can be given by the first two estimators bellow 2.9, 2.10 ([PAL05]). But I kept this one 2.11 ([SS94], [Ala98]).

This document and the information contained herein are the property of Matthieu PHILIPPE.
They must not be copied or disclosed to third parties without prior written authorization.

$$F_m = \frac{1}{2}.C_m.\rho.\mu^2 \quad (2.8)$$

$$C_m = \frac{r\omega}{V} \quad (2.9)$$

$$C_m = -0,05 + \sqrt{0,0025 + 0,36\left(\frac{r\omega}{V}\right)} \quad (2.10)$$

$$C_m = 0,5.\rho(T).S.R^2.C_l.\mid \frac{\omega}{V} \mid^{0,4}.V \quad (2.11)$$

$$C_l = \frac{1}{2 + (V/R\omega) * sin(\zeta)} \quad (2.12)$$

[GAS] gives a correction or adjustment coefficient C_l (2.12) to (2.11), where ζ in Lift Coefficient, is the angle between the axis of rotation and the direction of motion. If $R\omega \ll V$, then $C_l \simeq R\omega\frac{sin\zeta}{V}$. Certainly more appropriate than (2.9) to take into account an effect of side spin and backspin.

$$sin\zeta = \frac{\parallel \overrightarrow{\omega} \times \overrightarrow{V} \parallel}{\omega.V} = \frac{1}{\omega.V}.\sqrt{((\omega_j.V_k - \omega_k.V_j)^2 + (\omega_k.V_i - \omega_i.V_k)^2 + (\omega_i.V_j - \omega_j.V_i)^2)} \quad (2.13)$$

In order to meet my needs, I have choosen to provide my adapted C_l values to (2.11) instead of (2.12), from tests with my own balls (texture and number of dipples impact Reynolds number (2.3) as shown in [ASC+11]). These data are recorded in a Scilab table C_{l1}. C_m becomes (2.14).

$$C_m = 0,5.\rho(T).S.R^2.C_{l1}.\mid \frac{\omega}{V} \mid^{0,4}.V \quad (2.14)$$

My own C_{l1} from Club Mizuno JPX and Decathlon's ball are given as example in Table 2.1.

Club	C_{l1}
D	0.64
W5	0.65
H3	0.54
I5	0.54
I6	0.83
I7	0.42
I8	0.53
I9	0.53
PW	0.52
AW	0.52
SW	0.51
LW	0.51

Table 2.1: Table of my C_{l1}

Remark 2: Knowing 2.7 and 2.14 and 2.1, I chose the following Magnus force model	$\overrightarrow{F_m} = \begin{pmatrix} C_m * (\omega_j * v_k - \omega_k * v_j) \\ C_m * (\omega_k * v_i - \omega_i * v_k) \\ C_m * (\omega_i * v_j - \omega_j * v_i) \end{pmatrix}$ (2.15)

This document and the information contained herein are the property of Matthieu PHILIPPE.
They must not be copied or disclosed to third parties without prior written authorization.

2.3 Gravity Force

Definition 3: The gravity force		*The gravity force is determined by the acceleration g to the ground.*

The gravity Force expression is simply,

Remark 3: The Gravity Force		$\overrightarrow{F_g} = \begin{pmatrix} 0 \\ -mg \\ 0 \end{pmatrix}$ (2.16)

2.4 Launch Velocity

Due to the impact of the golf club to the ball, it results a ball flight with a launch angle. The club head loft determines the launch angle, the club head speed and the texture of both determine the launch ball speed and ball spin [Pen01].

Due to the texture we have a loss in energy, that can be taken into account throught the Coefficient Of Restitution *e* (COR [LJ94]).

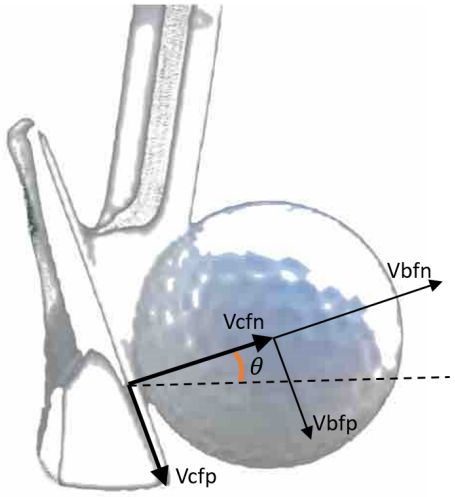


Figure 2.2: Velocity Components

[Pen01] gives an estimation of COR e :

$$e = 0,86 - 0,0029.V_{ci}.cos(\theta) \quad (2.17)$$

$$V_{bfn} = V_{ci}.cos(\theta).\frac{1+e}{1+\frac{m}{M}} \quad (2.18)$$

$$V_{bfp} = -V_{ci}.\frac{sin(\theta)}{1+\frac{m}{M}+\frac{5}{2}} \quad (2.19)$$

$$\psi_{bo} = \theta + \arctan\left(\frac{V_{bfp}}{V_{bfn}}\right) \quad (2.20)$$

$$\omega_{bo} = \omega_{bf} = -m.V_{bfp}.\frac{R}{I} \quad (2.21)$$

This document and the information contained herein are the property of Matthieu PHILIPPE.
They must not be copied or disclosed to third parties without prior written authorization.

where,

$$\begin{aligned} V_{ci} &= Initial\ Club\ Velocity,\ parallel\ to\ the\ ground. \\ V_{bfn} &= Final\ Ball\ Velocity\ Orthogonal\ to\ golf\ club\ face \\ V_{bfp} &= Velocity\ of\ the\ golf\ ball,\ parallel\ of\ the\ club\ face \\ \theta &= dynamic\ loft\ (\ club\ loft + Shaft\ Lean\ Impact\ <\ loft) \\ e &= COR,\ e \simeq 0.78\ for\ Irons,\ e \simeq 0.83\ for\ Drives\ and\ Woods \\ \psi_{bo} &= Launch\ Angle \\ \omega_{bo} &= Spin\ of\ the\ golf\ ball \\ I &= Inertia\ (2.3) \end{aligned}$$

2.5 Balls Velocity with Miss sweet spot

It is difficult to perfectly center the ball and the club. The ball velocity model is modified to include a *Miss* parameter. It is the amount by which the sweet spot is missed, the initial ball velocity in this case is (2.22),

$$V_{bi} = (1 - 35.56 * miss).\sqrt{V_{bfn}^2 + V_{bfp}^2}\ (miss\ mesured\ in\ meter) \tag{2.22}$$

2.6 Club Face Initial Velocity

Due to the Club Path (α_{cp}) and the Face to Path and Face Angle (γ_{fp}), the initial ball velocity and spin are modified.

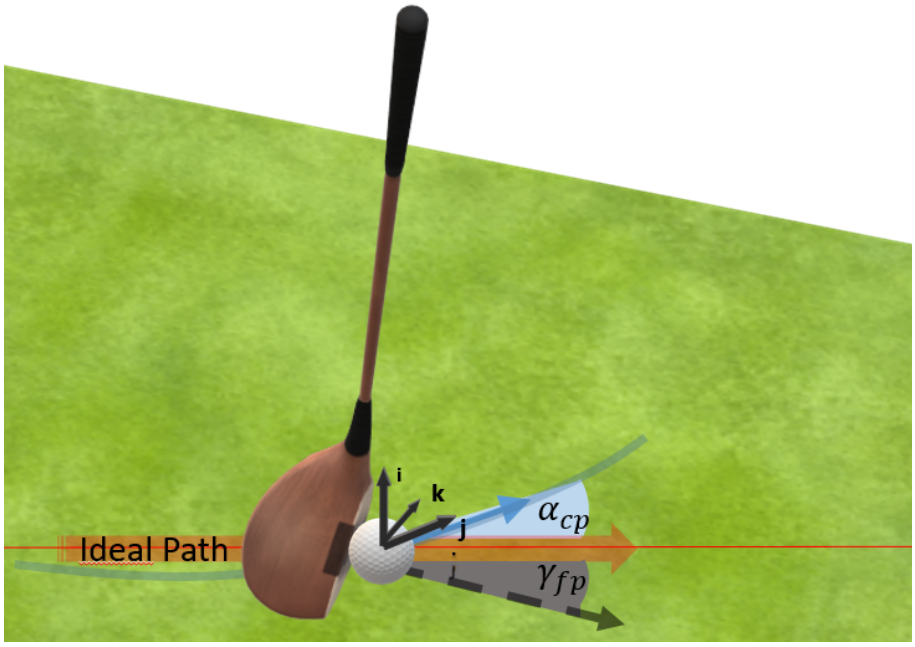


Figure 2.3: Club Path

Ainsi :

This document and the information contained herein are the property of Matthieu PHILIPPE.
They must not be copied or disclosed to third parties without prior written authorization.

$$V_{b0} \cdot \mathbf{i} = V_{bi} \cdot (1 - 0.3556 \cdot \text{miss}) \cdot \cos\left(\theta + \arctan\left(\frac{\sin(\theta) \cdot (1 + \frac{m}{M})}{(1+e) \cdot \cos(\theta) \cdot (1 + \frac{m}{M} + \frac{5}{2})}\right)\right)$$

$$V_{b0} \cdot \overrightarrow{j} = V_{bi} \cdot (1 - 0.3556 \cdot \text{miss}) \cdot \sin\left(\theta + \arctan\left(\frac{\sin(\theta) \cdot \left(1 + \frac{m}{M}\right)}{(1+e) \cdot \cos(\theta) \cdot \left(1 + \frac{m}{M} + \frac{5}{2}\right)}\right)\right)$$

$$V_{b0} \cdot \overrightarrow{k} = 0$$

The spin about the $\overrightarrow{i}$ axis is not a component along the club path axis. Spin is applied in the ground plane, with *SideSpin* aligned along the club face axis $\overrightarrow{j}$, and *BackSpin* aligned along the club path axis $\overrightarrow{k}$. $w_{\overrightarrow{i}} = 0$

$$w_{\overrightarrow{j}} = V_{b0} \cdot \overrightarrow{j} \cdot \sin\left(\alpha_{cp} - \gamma_{fp}\right) \cdot \text{Sac.coeffSpinLift(Sac.Type == Club)}$$

$$w_{\overrightarrow{k}} = V_{b0} \cdot \overrightarrow{i} \cdot \sin(\theta) \cdot \text{Sac.coeffSpin(Sac.Type == Club)}$$

2.7 Complete Model of flight

2.7.1 Complete Model of Flight (Matrix/Vector Form)

The golf ball's flight is described by the following ODE system for the state vector $\vec{Y} = \begin{pmatrix} x & y & z & V_x & V_y & V_z \end{pmatrix}^T$:

$$\frac{d\vec{Y}}{dt} = \begin{pmatrix} V_x \\ V_y \\ V_z \\ -\frac{1}{2m}\rho(T)C_d S\|\vec{V}\|V_x + \frac{C_m}{m}(\omega_y V_z - \omega_z V_y) \\ -\frac{1}{2m}\rho(T)C_d S\|\vec{V}\|V_y + \frac{C_m}{m}(\omega_z V_x - \omega_x V_z) - g \\ -\frac{1}{2m}\rho(T)C_d S\|\vec{V}\|V_z + \frac{C_m}{m}(\omega_x V_y - \omega_y V_x) \end{pmatrix} \tag{2.23}$$

where:

- $\vec{V} = \begin{pmatrix} V_x \\ V_y \\ V_z \end{pmatrix}$ is the velocity vector,
- $\vec{\omega} = \begin{pmatrix} \omega_x \\ \omega_y \\ \omega_z \end{pmatrix}$ is the spin vector,
- $\|\vec{V}\| = \sqrt{V_x^2 + V_y^2 + V_z^2}$,
- C_d is the drag coefficient (Equation 2.4),
- $C_m = 0.5\rho(T)SR^2C_{l1}\left(\frac{\|\vec{\omega}\|}{\|\vec{V}\|}\right)^{0.4}\|\vec{V}\|$ (Equation 2.14).

This document and the information contained herein are the property of Matthieu PHILIPPE.
They must not be copied or disclosed to third parties without prior written authorization.

2.7.1.1 Initial Conditions

The initial state vector $\vec{Y}_0$ is:

$$\vec{Y}_0 = \begin{pmatrix} 0 \\ 0 \\ 0 \\ V_{bi}\cos\left(\theta + \arctan\left(\frac{\sin\theta\cdot(1+\frac{m}{M})}{(1+e)\cos\theta\cdot(1+\frac{m}{M}+\frac{5}{2})}\right)\right) \\ V_{bi}\sin\left(\theta + \arctan\left(\frac{\sin\theta\cdot(1+\frac{m}{M})}{(1+e)\cos\theta\cdot(1+\frac{m}{M}+\frac{5}{2})}\right)\right) \\ 0 \end{pmatrix} \tag{2.24}$$

with $V_{bi} = (1-35.56\cdot\text{miss})\sqrt{V_{bfn}^2 + V_{bfp}^2}$ and $\vec{\omega}_0 = \begin{pmatrix} 0 & V_{b0y}\sin\left(\alpha_{cp} - \gamma_{fp}\right)\cdot\text{SpinLiftCoeff} & V_{b0x}\sin(\theta)\cdot\text{SpinCoeff} \end{pmatrix}^T$.

2.7.2 Complete Model of Flight (Tensor Notation)

The golf ball's flight dynamics are described using tensor notation as follows:

2.7.2.1 Governing Equation

The time evolution of the state vector $\vec{Y} = \begin{pmatrix} \vec{r} \\ \vec{V} \end{pmatrix} \in \mathbb{R}^6$ is given by:

$$\frac{d\vec{Y}}{dt} = \begin{pmatrix} \vec{V} \\ \frac{1}{m}\left(\vec{F}_d + \vec{F}_m + \vec{F}_g\right) \end{pmatrix} \tag{2.25}$$

2.7.2.2 Drag Force

The drag force is a nonlinear vector field:

$$\vec{F}_d = -\frac{1}{2}\rho(T)\,C_d(\|\vec{V}\|, Re)\,S\,\|\vec{V}\|\,\vec{V}, \quad \|\vec{V}\| = \sqrt{\vec{V}\cdot\vec{V}} \tag{2.26}$$

where the drag coefficient C_d depends on the Reynolds number $Re = \frac{\rho(T)\|\vec{V}\|D}{\mu}$:

$$C_d = 0.36 + \frac{24}{Re} + \frac{6}{1+\sqrt{Re}}. \tag{2.27}$$

2.7.2.3 Magnus Force

The Magnus force is expressed using the cross product:

$$\vec{F}_m = C_m(\vec{\omega}\times\vec{V}), \quad C_m = \frac{1}{2}\rho(T)\,S\,R^2\,C_{l1}\left(\frac{\|\vec{\omega}\|}{\|\vec{V}\|}\right)^{0.4}\|\vec{V}\| \tag{2.28}$$

where $\vec{\omega}\times\vec{V}$ is the antisymmetric cross product, representable via the Levi-Civita symbol ϵ_{ijk}:

$$(\vec{\omega}\times\vec{V})_i = \sum_{j,k}\epsilon_{ijk}\,\omega_j\,V_k. \tag{2.29}$$

2.7.2.4 Gravity Force

$$\vec{F}_g = -mg\,\hat{e}_y, \quad \hat{e}_y = \begin{pmatrix} 0 \\ 1 \\ 0 \end{pmatrix}. \tag{2.30}$$

This document and the information contained herein are the property of Matthieu PHILIPPE.
They must not be copied or disclosed to third parties without prior written authorization.

2.7.2.5 Initial Conditions

The initial state vector $\vec{Y}_0$ is:

$$\vec{Y}_0 = \begin{pmatrix} \vec{0} \\ \vec{V}_{b0} \end{pmatrix}, \quad \vec{V}_{b0} = V_{bi}\,\hat{u}(\theta, e, m, M) \tag{2.31}$$

where $V_{bi} = (1 - 35.56 \cdot \text{miss})\sqrt{V_{bfn}^2 + V_{bfp}^2}$ and the unit direction vector $\hat{u}$ is:

$$\hat{u} = \begin{pmatrix} \cos\left(\theta + \arctan\left(\frac{\sin\theta\cdot(1+\frac{m}{M})}{(1+e)\cos\theta\cdot(1+\frac{m}{M}+\frac{5}{2})}\right)\right) \\ \sin\left(\theta + \arctan\left(\frac{\sin\theta\cdot(1+\frac{m}{M})}{(1+e)\cos\theta\cdot(1+\frac{m}{M}+\frac{5}{2})}\right)\right) \\ 0 \end{pmatrix}. \tag{2.32}$$

The initial spin vector $\vec{\omega}_0$ is:

$$\vec{\omega}_0 = \begin{pmatrix} 0 \\ V_{b0y}\sin\left(\alpha_{cp} - \gamma_{fp}\right)\cdot \text{SpinLiftCoeff} \\ V_{b0x}\sin(\theta)\cdot \text{SpinCoeff} \end{pmatrix}. \tag{2.33}$$

2.8 External Function for the Ordinary Differential Equation solver

The simulation of the golf ball's trajectory relies on solving a system of **ordinary differential equations (ODEs)** that model the physical forces acting on the ball during flight, bounce, and roll. This section details the design and implementation of the external functions used to define these ODEs, as well as their integration with the numerical solver. The approach leverages modularity to separate the physical modeling from the numerical resolution, enabling flexibility and maintainability.

2.8.1 Architecture and Design

The ODE solver architecture is structured around **three core components**:

- **Equation Definition**: External functions (e.g., `flightg` for flight and `rolling` for roll) define the system of ODEs based on physical laws.
- **Event Detection**: Functions like `events` and `events2` handle discrete events (e.g., ball impact with the ground or stopping condition).
- **Numerical Integration**: The `ode` function in Scilab (or equivalent in other languages) performs the time integration using a specified method (e.g., Runge-Kutta 4th order).

This separation ensures that the physical model can be modified independently of the numerical solver, which is critical for iterative development and validation.

2.8.2 Equation Definition Functions

The ODEs are defined in external functions that return the time derivatives of the state variables. For example, the `flightg` function in Scilab computes the derivatives for the flight phase, incorporating the following forces:

- **Drag Force**: The drag force is modeled using the **White's formula** for the drag coefficient C_d, which depends on the Reynolds number Re:

$$C_d = 0.36 + \frac{24}{\text{Re}} + \frac{6}{1+\sqrt{\text{Re}}}, \quad \text{where} \quad \text{Re} = \frac{V \cdot D}{\nu}$$

  Here, V is the ball's velocity, D is the ball's diameter, and ν is the kinematic viscosity of air ($1.5 \times 10^{-5}\,\text{m}^2/\text{s}$).

This document and the information contained herein are the property of Matthieu PHILIPPE.
They must not be copied or disclosed to third parties without prior written authorization.

- **Magnus Force**: The Magnus force, responsible for the lift due to spin, is computed using the **Smits and Smith model** [?]:

$$C_m = C_{l1} \cdot r \cdot \left| \frac{\omega}{V} \right|^{0.4}$$

where C_{l1} is an empirical coefficient, r is the ball's radius, ω is the spin rate, and V is the ball's velocity. The Magnus force is then:

$$\mathbf{F}_m = \frac{1}{2} \rho C_m A r (\boldsymbol{\omega} \times \mathbf{v})$$

where ρ is the air density, A is the ball's cross-sectional area, and $\mathbf{v}$ is the velocity vector.

- **Gravity**: The gravitational force is simply $\mathbf{F}_g = (0, 0, -mg)$, where g is the gravitational acceleration.

The state vector for the flight phase includes the ball's position (x, y, z), velocity (v_x, v_y, v_z), and spin $(\omega_x, \omega_y, \omega_z)$. The ODEs are thus a system of 9 coupled equations:

$$\frac{d}{dt} \begin{bmatrix} x \\ v_x \\ y \\ v_y \\ z \\ v_z \\ \omega_x \\ \omega_y \\ \omega_z \end{bmatrix} = \begin{bmatrix} v_x \\ \frac{F_{dx} + F_{mx}}{m} \\ v_y \\ \frac{F_{dy} + F_{my}}{m} - g \\ v_z \\ \frac{F_{dz} + F_{mz}}{m} \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

Note that the current implementation assumes **no spin decay** during flight ($\frac{d\omega}{dt} = 0$), which is a simplification that may introduce errors over long distances (see Section 2.8.5).

2.8.3 Event Detection

Discrete events, such as the ball's impact with the ground or its stopping condition, are handled using **event functions**. These functions are critical for:

- Detecting when the ball hits the ground ($y \leq \text{Altitude}(x, z)$).
- Triggering the transition from flight to bounce or roll phases.
- Ensuring numerical stability by adapting the time step near event points.

In Scilab, the `events` function checks for the ball's impact with the ground:

```
function [value] = events(t, x)
    if (x(3) < AltiSol(x(1), x(5))) then
        value = 0;
    else
        value = x(3) - AltiSol(x(1), x(5));
    end
endfunction
```

Here, `x(3)` represents the ball's height y, and `AltiSol` computes the ground altitude at position (x, z). The event is triggered when the ball's height falls below the ground level.

Similarly, the `events2` function detects when the ball stops rolling by checking if its horizontal velocity v_x falls below a threshold:

```
function [value] = events2(t, x)
    value = x(2) - 0; // Stop if v_x <= 0
endfunction
```

This document and the information contained herein are the property of Matthieu PHILIPPE.
They must not be copied or disclosed to third parties without prior written authorization.

Limitations:

- The current event detection uses a **fixed threshold** (e.g., 10^{-4}), which may not be optimal for all scenarios. A dynamic threshold based on the ball's velocity or spin could improve robustness.
- The event functions do not account for **latitudinal motion** (v_y) during rolling, which may lead to inaccuracies in cases where the ball has a significant side spin.

2.8.4 Numerical Integration

The ODE system is integrated numerically using Scilab's `ode` function, which supports various methods such as **Runge-Kutta 4th order (RK4)**. The integration is performed with a fixed time step Δt, which is set to $0.1\,\text{s}$ in the current implementation.

```
[Pos, rd] = ode('roots', Init0, T0, 0:0.1:Tfin, flightg, 1, events);
```

Here:

- `Init0` is the initial state vector.
- `T0` is the initial time.
- `0:0.1:Tfin` defines the time grid with a step size of $0.1\,\text{s}$.
- `flightg` is the function defining the ODEs for the flight phase.
- `events` is the event function for detecting impacts.

Strengths:

- The use of a **standard numerical method** (RK4) ensures stability and accuracy for most scenarios.
- The modular design allows for easy replacement of the solver (e.g., switching to an adaptive method like RK).

Limitations:

- The **fixed time step** ($\Delta t = 0.1\,\text{s}$) may be too large to capture rapid dynamics, such as the initial phase of the flight or the bounce. A smaller step (e.g., $0.01\,\text{s}$) or an adaptive method would improve precision.
- The solver does not account for **stiffness** in the ODE system, which could arise in certain edge cases (e.g., very high spin rates). Implicit or semi-implicit methods (e.g., BDF) may be more suitable for such scenarios.

2.8.5 Limitations and Potential Evolutions

While the current implementation is robust and validated experimentally, several limitations and areas for improvement have been identified:

- **Spin Decay**: The current model assumes that the spin rate remains constant during flight ($\frac{d\boldsymbol{\omega}}{dt} = 0$). In reality, spin decays due to air resistance, which affects the Magnus force and thus the trajectory. Adding spin decay terms to the ODEs would improve accuracy, particularly for long-distance shots (e.g., driver shots).
  - **Proposed Evolution**: Introduce a spin decay model based on the torque due to air resistance:

$$\frac{d\boldsymbol{\omega}}{dt} = -\frac{k\rho A r \boldsymbol{\omega} V}{I}$$

This document and the information contained herein are the property of Matthieu PHILIPPE.
They must not be copied or disclosed to third parties without prior written authorization.

where k is an empirical coefficient, I is the ball's moment of inertia, and V is the ball's velocity.

- **Dynamic Drag and Magnus Coefficients**: The drag coefficient C_d and Magnus coefficient C_m are currently modeled using simplified formulas. While these are reasonable approximations, they do not fully capture the complexities of a golf ball's aerodynamics, particularly the effect of dimples.
  - **Proposed Evolution**: Use experimental data or CFD simulations to derive more accurate coefficients for C_d and C_m as functions of Re and spin rate.
- **Event Detection Robustness**: The current event detection relies on fixed thresholds, which may not be optimal for all scenarios. For example, the threshold for detecting ground impact (10^{-4}) may be too large for high-precision simulations.
  - **Proposed Evolution**: Implement a dynamic threshold or use interpolation to precisely locate the event time between time steps.
- **Adaptive Time Stepping**: The fixed time step ($\Delta t = 0.1\,\text{s}$) may not be efficient for all phases of the simulation. For instance, a smaller step is needed during the bounce phase, while a larger step may suffice during the roll phase.
  - **Proposed Evolution**: Replace the fixed-step RK4 method with an adaptive method (e.g., RK45 or Dormand-Prince) that adjusts the time step based on the local truncation error.
- **Lateral Motion in Bounces**: The current bounce model does not account for lateral motion (v_y) or side spin (ω_z) during the bounce. This limits the model's ability to accurately simulate slices and hooks.
  - **Proposed Evolution**: Extend the bounce model to include lateral components, using a 3D restitution model that accounts for both vertical and horizontal velocities, as well as spin.
- **Terrain-Specific Parameters**: The friction coefficients (μ) and rolling resistance (ρ_g) are currently constant. In reality, these parameters depend on the terrain type (e.g., green, fairway, rough).
  - **Proposed Evolution**: Introduce terrain-specific parameters and allow the user to select the terrain type for each simulation.

These evolutions would enhance the model's accuracy and robustness, particularly for edge cases and high-precision applications. However, they should be implemented incrementally and validated against experimental data to ensure that the improvements do not introduce new errors.

This document and the information contained herein are the property of Matthieu PHILIPPE.
They must not be copied or disclosed to third parties without prior written authorization.

Chapter 3

Bouncing and Rolling

3.1 Bouncing and Rolling

The dynamics of a golf ball after impact with the ground are governed by complex interactions between the ball's velocity, spin, and the surface properties. This section details the modeling of the **bouncing** and **rolling** phases, based on the work of Woo-Jin Roh and Chong-Won Lee [RL10] and the experimental validation of the Scilab implementation. The transition from flight to bounce and then to roll is critical for accurately predicting the ball's final position, particularly for short-game shots where these phases dominate the total distance.

3.1.1 Ball's Characteristics During Bouncing

The bouncing phase begins when the ball impacts the ground, detected using the event function described in Section 2.8.3. The model for bouncing is based on the **restitution coefficients** and **friction forces** acting during the short duration of the impact. The key parameters and equations are derived from [RL10], with adaptations to fit the experimental data.

3.1.1.1 Impact Angle and Restitution

The impact angle θ_{impact} is defined as the angle between the ball's velocity vector and the ground at the moment of impact. In the implementation, this is computed as:

$$\theta_{\text{impact}} = \arctan\left(\frac{|v_{ix}|}{|v_{iy}|}\right),$$

where:

- $v_{ix} = \sqrt{v_x^2 + v_z^2}$ is the horizontal component of the velocity (in the plane of the ground).
- v_{iy} is the vertical component of the velocity.

The **restitution coefficient** e_{ball} determines the ratio of the relative velocity after the impact to the relative velocity before the impact. In the model, e_{ball} is empirically defined as a function of the vertical velocity component v'_{iy}:

$$e_{\text{ball}} = \begin{cases} 0.510 - 0.0375 \cdot |v'_{iy}| + 0.000903 \cdot |v'_{iy}|^2 & \text{if } |v'_{iy}| < 20\,\text{m/s}, \\ 0.120 & \text{otherwise.} \end{cases}$$

This piecewise function captures the **nonlinear behavior** of the ball's restitution, where higher impact velocities result in lower restitution coefficients due to energy loss from deformation and other dissipative effects.

This document and the information contained herein are the property of Matthieu PHILIPPE.
They must not be copied or disclosed to third parties without prior written authorization.

Critique:

- The empirical coefficients (0.510, -0.0375, 0.000903) are specific to the **green turf** conditions tested in [RL10]. These may not be universally applicable to other surfaces (e.g., fairway, rough, or sand).
- The threshold of $20\,\text{m/s}$ is arbitrary and may not hold for all golf ball models or surface types. Experimental validation for different conditions is recommended.

—

3.1.1.2 Bounce Angle and Velocity Update

The angle of the ball's trajectory after the bounce, θ_{rebond}, is computed using an empirical relationship from [RL10]:

$$\theta_{\text{rebond}} = 15.4 \cdot \frac{|\vec{v_i}|}{18.6} \cdot \frac{\theta_{\text{impact}}}{44.4},$$

where $|\vec{v_i}|$ is the magnitude of the ball's velocity at impact. This equation assumes that the bounce angle is proportional to both the impact angle and the velocity magnitude, scaled by empirical constants.

The post-impact velocities are then calculated as:

$$v'_{ix} = |\vec{v_i}|\cos(\theta_{\text{rebond}}) - |v_{iy}|\sin(\theta_{\text{rebond}}),$$
$$v'_{iy} = |\vec{v_i}|\sin(\theta_{\text{rebond}}) + |v_{iy}|\cos(\theta_{\text{rebond}}).$$

Critique:

- The empirical constants (15.4, 18.6, 44.4) are derived from experiments on specific turf types and may not generalize well to other surfaces. For example, a harder surface (e.g., fairway) would likely result in a smaller θ_{rebond} for the same impact conditions.
- The model assumes a **2D bounce** (ignoring lateral motion v_y). This simplification may lead to inaccuracies for shots with significant side spin or lateral velocity components (e.g., slices or hooks).

Proposed Improvements:

- **Terrain-Specific Parameters**: Introduce a terrain-dependent scaling factor for θ_{rebond}. For example:

  $$\theta_{\text{rebond}} = k_{\text{terrain}} \cdot 15.4 \cdot \frac{|\vec{v_i}|}{18.6} \cdot \frac{\theta_{\text{impact}}}{44.4},$$

  where k_{terrain} is 1.0 for green, 0.8 for fairway, and 0.6 for rough.
- **3D Bounce Model**: Extend the model to include lateral motion and side spin. For example, the post-impact lateral velocity v'_{iy} could be updated as:

  $$v'_{iy} = e_{\text{lat}} \cdot v_{iy} \cdot \text{sign}(v_y),$$

  where e_{lat} is a lateral restitution coefficient (typically 0.8–0.9 for green).

3.1.1.3 Friction and Spin During Bounce

During the bounce, friction between the ball and the ground affects both the **linear velocity** and the **spin** of the ball. The model uses a **Coulomb friction** approach, where the friction force is proportional to the normal

This document and the information contained herein are the property of Matthieu PHILIPPE.
They must not be copied or disclosed to third parties without prior written authorization.

force and the coefficient of friction μ. The critical friction oefficient μ_{critic} determines whether the ball **slides** or **rolls without slipping** after the bounce:

$$\mu_{\text{critic}} = \frac{2\left(|v'_{ix}| + r|\omega_i|\right)}{7(1 + e_{\text{ball}})|v'_{iy}|},$$

where:

- r is the ball's radius.
- ω_i is the initial spin rate (backspin or side spin).

If $\mu < \mu_{\text{critic}}$, the ball slides, and the post-impact velocities and spin are updated as:

$$\begin{aligned} v'_{rx} &= v'_{ix} - \mu|v'_{iy}|(1 + e_{\text{ball}}), \\ v'_{ry} &= e_{\text{ball}}|v'_{iy}|, \\ \omega_r &= \omega_i - 5\mu|v'_{iy}|(1 + e_{\text{ball}}). \end{aligned}$$

If $\mu \geq \mu_{\text{critic}}$, the ball rolls without slipping, and the velocities and spin are updated as:

$$\begin{aligned} v'_{rx} &= \frac{5}{7}v'_{ix} - \frac{2}{7}r\omega_i, \\ v'_{ry} &= e_{\text{ball}}|v'_{iy}|, \\ \omega_r &= -\frac{v'_{rx}}{r}. \end{aligned}$$

Critique:

- The coefficient of friction $\mu = 0.40$ is a **fixed value** for all surfaces. In reality, μ varies significantly with terrain type (e.g., 0.2–0.3 for fairway, 0.5–0.7 for rough).
- The model assumes that the **side spin** (ω_z) does not affect the bounce dynamics, which is a simplification. In reality, side spin can induce lateral forces during the bounce, particularly on uneven surfaces.

Proposed Improvements:

- **Dynamic Friction Coefficients**: Use terrain-specific values for μ and μ_{critic}. For example:

$$\mu = \begin{cases} 0.40 & \text{for green,} \\ 0.20 & \text{for fairway,} \\ 0.60 & \text{for rough.} \end{cases}$$

- **Side Spin in Bounce**: Extend the model to include the effect of side spin on lateral motion. For example, the lateral velocity after bounce could be updated as:

$$v'_{ry} = e_{\text{lat}} \cdot v_{iy} + k_{\text{spin}} \cdot r\omega_z,$$

where k_{spin} is an empirical coefficient (e.g., 0.1–0.2).

3.1.2 Ball's Characteristics During Rolling

Once the ball's vertical velocity is negligible and it remains in contact with the ground, the rolling phase begins. During this phase, the ball's motion is dominated by **friction** and **rolling resistance**. The model for rolling is based on the work of Penner [Pen02a, Pen02b], which assumes a linear relationship between the deceleration and the velocity.

This document and the information contained herein are the property of Matthieu PHILIPPE.
They must not be copied or disclosed to third parties without prior written authorization.

3.1.2.1 Rolling Resistance

The rolling resistance force is modeled as a **viscous friction** term, proportional to the ball's velocity:

$$\mathbf{F}_{\text{roll}} = -\frac{5}{7}\rho_g g \mathbf{v},$$

where:

- $\rho_g = 0.131$ is the rolling resistance coefficient for green turf.
- g is the gravitational acceleration.
- $\mathbf{v} = (v_x, v_y, v_z)$ is the velocity vector.

The equations of motion for the rolling phase are:

$$\frac{d}{dt}\begin{bmatrix} x \\ v_x \\ y \\ v_y \\ z \\ v_z \end{bmatrix} = \begin{bmatrix} v_x \\ -\frac{5}{7}\rho_g g \frac{v_x}{\|\mathbf{v}\|} \\ v_y \\ -\frac{5}{7}\rho_g g \frac{v_y}{\|\mathbf{v}\|} \\ v_z \\ -\frac{5}{7}\rho_g g \frac{v_z}{\|\mathbf{v}\|} \end{bmatrix}.$$

Critique:

- The rolling resistance coefficient $\rho_g = 0.131$ is **fixed** and specific to green turf. For other surfaces, ρ_g can vary significantly (e.g., 0.065–0.196 for fairway [Pen02b]).
- The model assumes that the ball **slides** until it stops, which is

  not physically accurate. In reality, the ball transitions to **rolling without slipping** when the friction force is sufficient to prevent sliding.

Proposed Improvements:

- **Terrain-Specific Rolling Resistance**: Use terrain-dependent values for ρ_g. For example:

$$\rho_g = \begin{cases} 0.131 & \text{for green,} \\ 0.10 & \text{for fairway,} \\ 0.30 & \text{for rough.} \end{cases}$$

- **Rolling Without Slipping**: Add a condition to transition to rolling without slipping when the friction force balances the rolling resistance. For example:

$$\text{If } |\mathbf{v}| < v_{\text{roll}}, \text{ then } \frac{d\mathbf{v}}{dt} = 0,$$

  where v_{roll} is the velocity threshold for rolling without slipping (e.g., 0.1 m/s).

3.1.2.2 Stopping Condition

The rolling phase ends when the ball's horizontal velocity falls below a threshold, detected using the `events2` function:

$$\text{value} = v_x - 0.$$

This document and the information contained herein are the property of Matthieu PHILIPPE.
They must not be copied or disclosed to third parties without prior written authorization.

This condition triggers the end of the simulation when $v_x \leq 0$.

Critique:

- The stopping condition only checks the **horizontal velocity** v_x and ignores v_y and v_z. This may lead to premature termination of the simulation if the ball has a significant lateral or vertical velocity component.
- The threshold of 0 is arbitrary and may not capture the ball's motion accurately for all scenarios.

Proposed Improvements:

- **3D Stopping Condition**: Use the magnitude of the velocity vector as the stopping criterion:

$$\text{value} = \|\mathbf{v}\| - v_{\text{stop}},$$

  where $v_{\text{stop}} = 10^{-4}\,\text{m/s}$ is a small threshold.

- **Dynamic Threshold**: Use a dynamic threshold based on the ball's spin or the surface roughness. For example:

$$v_{\text{stop}} = k_{\text{stop}} \cdot r\omega,$$

  where k_{stop} is an empirical coefficient (e.g., 0.01).

3.1.3 Limitations and Potential Evolutions

While the bouncing and rolling model is based on well-established physical principles and has been validated experimentally, several limitations and areas for improvement have been identified:

- **Fixed Coefficients**: The restitution coefficient e_{ball}, friction coefficient μ, and rolling resistance ρ_g are currently fixed. These coefficients depend on the **terrain type**, **ball model**, and **impact conditions**.
  - **Proposed Evolution**: Introduce a **terrain-specific parameter set** and allow the user to select the terrain type for each simulation. For example, create a lookup table for e_{ball}, μ, and ρ_g based on the surface (green, fairway, rough, sand).
- **2D Bounce Model**: The current bounce model assumes a 2D motion (ignoring lateral components v_y and ω_z). This limits the model's ability to accurately simulate **slices** and **hooks**.
  - **Proposed Evolution**: Extend the bounce model to 3D by including lateral motion and side spin. For example, use a 3D restitution model that accounts for all velocity and spin components:

$$\begin{aligned}
v'_{ix} &= e_{\text{long}} v_{ix} - \mu(1 + e_{\text{long}})|v_{iy}|, \\
v'_{iy} &= e_{\text{vert}} v_{iy}, \\
v'_{iz} &= e_{\text{lat}} v_{iz} + k_{\text{spin}} r\omega_z, \\
\omega'_x &= \omega_x, \\
\omega'_y &= \omega_y - \frac{5}{2}\mu(1 + e_{\text{vert}})|v_{iy}|, \\
\omega'_z &= \omega_z - \frac{5}{2}\mu(1 + e_{\text{lat}})|v_{iz}|,
\end{aligned}$$

    where e_{long}, e_{vert}, and e_{lat} are the longitudinal, vertical, and lateral restitution coefficients, respectively.
- **Spin Decay During Rolling**: The current model assumes that the spin remains constant during rolling. In reality, spin decays due to **rolling resistance** and **friction**.

This document and the information contained herein are the property of Matthieu PHILIPPE.
They must not be copied or disclosed to third parties without prior written authorization.

- **Proposed Evolution**: Add a spin decay term to the rolling equations. For example:

$$\frac{d\omega}{dt} = -\frac{k\rho_g g r \omega}{I},$$

where k is an empirical coefficient (e.g., 0.1) and I is the ball's moment of inertia.

- **Surface Roughness and Slope**: The current model assumes a **flat and uniform surface**. In reality, the surface may have **slope** or **roughness**, which can affect the ball's motion.
  - **Proposed Evolution**: Incorporate a **slope model** into the rolling equations. For example, if the surface has a slope α in the x-direction, the gravitational force component along the slope is:

$$F_{g,x} = mg\sin(\alpha).$$

This force can be added to the rolling resistance term.

- **Validation for Different Balls**: The current model uses empirical coefficients derived from specific ball models (e.g., Mizuno JPX, Decathlon). These coefficients may not be applicable to all golf balls.
  - **Proposed Evolution**: Conduct experiments to derive **ball-specific coefficients** for e_{ball}, μ, and ρ_g. Store these coefficients in a lookup table indexed by ball model.

These evolutions would significantly enhance the accuracy and robustness of the bouncing and rolling model, particularly for edge cases such as high-spin shots, uneven surfaces, or non-standard ball models. However, each improvement should be **validated against experimental data** to ensure that it does not introduce new errors or inconsistencies.

3.2 Scilab Code Explanation for Golf Ball Simulation

This section provides a detailed explanation of the Scilab code used to simulate the flight, bounce, and roll of a golf ball. The code is designed with simplifications to ensure rapid computation, as it is intended to process real-time radar data of the ball's initial flight (which will be discussed in the next chapter).

—

3.2.1 The `flightg` Function

The `flightg` function is the core of the flight phase simulation. It computes the time derivatives of the state variables (position, velocity, and spin) based on the physical forces acting on the ball: drag, Magnus lift, and gravity.

3.2.1.1 Inputs and Outputs

- **Inputs**:
  - `t`: Current time (s).
  - `v`: State vector of size 12 containing:
    - `v(1)`: Position x (m).
    - `v(2)`: Velocity v_x (m/s).
    - `v(3)`: Position y (m).

This document and the information contained herein are the property of Matthieu PHILIPPE.
They must not be copied or disclosed to third parties without prior written authorization.

  * v(4): Velocity v_y (m/s).
  * v(5): Position z (m).
  * v(6): Velocity v_z (m/s).
  * v(7): Spin ω_x (rad/s).
  * v(8): Spin ω_y (rad/s).
  * v(9): Spin ω_z (rad/s).
  * v(10): Magnus coefficient C_m.
  * v(11): Mass m (kg).
  * v(12): Diameter D (m).
- **Output**:
  - `vprime`: Vector of time derivatives of the state variables.

3.2.1.2 Internal Logic

The `flightg` function follows these steps to compute the derivatives:

1. **Physical Constants Initialization**:
   - The mass m and diameter D are extracted from the state vector.
   - The air density ρ_{air} is calculated based on a fixed temperature $T = 20°C$:
   $$\rho_{\text{air}} = 1.292 \times \frac{273.15}{273.15 + T}$$
   - The cross-sectional area A of the ball is computed:
   $$A = \pi \left(\frac{D}{2}\right)^2$$
2. **Velocity and Spin Calculation**:
   - The velocity magnitude V_m is computed:
   $$V_m = \sqrt{v_x^2 + v_y^2 + v_z^2}$$
   - The spin magnitude W_m is computed:
   $$W_m = \sqrt{\omega_x^2 + \omega_y^2 + \omega_z^2}$$
3. **Drag Coefficient Calculation**:
   - The Reynolds number Re is computed:
   $$\text{Re} = \frac{V_m \times D}{1.5 \times 10^{-5}}$$

This document and the information contained herein are the property of Matthieu PHILIPPE.
They must not be copied or disclosed to third parties without prior written authorization.

- The drag coefficient C_d is computed using White's formula:

$$C_d = 0.36 + \frac{24}{\text{Re}} + \frac{6}{1 + \sqrt{\text{Re}}}$$

4. **Drag Force Calculation**:

- The magnitude of the drag force D is computed:

$$D = \frac{1}{2}\rho_{\text{air}} C_d A V_m^2$$

- The components of the drag force are computed based on the direction of velocity:

$$F_{dx} = -\frac{D \times v_x}{V_m}, \quad F_{dy} = -\frac{D \times v_y}{V_m}, \quad F_{dz} = -\frac{D \times v_z}{V_m}$$

5. **Magnus Force Calculation**:

- The magnitude of the Magnus force M is computed using the empirical coefficient C_{l1} stored in `v(10)`:

$$M = \frac{1}{2}\rho_{\text{air}} A \left(\frac{D}{2}\right) C_{l1} V_m$$

- The components of the Magnus force are computed using the cross product $\boldsymbol{\omega} \times \mathbf{v}$:

$$F_{mx} = M \times (\omega_y v_z - \omega_z v_y), \quad F_{my} = M \times (\omega_z v_x - \omega_x v_z), \quad F_{mz} = M \times (\omega_x v_y - \omega_y v_x)$$

6. **Time Derivatives Calculation**:

- The derivatives of the positions are the velocities:

$$\frac{dx}{dt} = v_x, \quad \frac{dy}{dt} = v_y, \quad \frac{dz}{dt} = v_z$$

- The derivatives of the velocities are computed using the drag, Magnus, and gravity forces:

$$\frac{dv_x}{dt} = \frac{F_{dx} + F_{mx}}{m}, \quad \frac{dv_y}{dt} = -g + \frac{F_{dy} + F_{my}}{m}, \quad \frac{dv_z}{dt} = \frac{F_{dz} + F_{mz}}{m}$$

- The derivatives of the spin components are assumed to be zero (no spin decay during flight for simplification):

$$\frac{d\omega_x}{dt} = \frac{d\omega_y}{dt} = \frac{d\omega_z}{dt} = 0$$

Note: The simplification of assuming no spin decay during flight is made to ensure rapid computation, as the model is designed to process real-time radar data of the ball's initial flight, which will be discussed in the next chapter.

—

3.2.2 The `rolling` Function

The `rolling` function models the rolling phase of the ball after it has landed. It computes the time derivatives of the state variables, taking into account friction and rolling resistance.

This document and the information contained herein are the property of Matthieu PHILIPPE.
They must not be copied or disclosed to third parties without prior written authorization.

3.2.2.1 Inputs and Outputs

- **Inputs**:
  - `t`: Current time (s).
  - `v`: State vector of size 10 containing:
    - `v(1)`: Position x (m).
    - `v(2)`: Velocity v_x (m/s).
    - `v(3)`: Position y (m).
    - `v(4)`: Velocity v_y (m/s).
    - `v(5)`: Position z (m).
    - `v(6)`: Velocity v_z (m/s).
    - `v(7)`: Spin ω_x (rad/s).
    - `v(8)`: Spin ω_y (rad/s).
    - `v(9)`: Spin ω_z (rad/s).
    - `v(10)`: Rolling resistance coefficient.
- **Output**:
  - `vprime`: Vector of time derivatives of the state variables.

3.2.2.2 Internal Logic

The `rolling` function follows these steps:

1. **Physical Constants Initialization**:
   - The mass m is fixed at $0.04545\,\text{kg}$.
   - The air density ρ_{air} is calculated for a temperature of $20°C$.
   - The rolling resistance coefficient ρ_g is fixed at 0.131 (typical value for a green).
   - The gravitational acceleration g is fixed at $9.81\,\text{m/s}^2$.
2. **Time Derivatives Calculation**:
   - The derivatives of the positions are the velocities:

$$\frac{dx}{dt} = v_x, \quad \frac{dy}{dt} = v_y, \quad \frac{dz}{dt} = v_z$$

   - The derivatives of the velocities are computed using a friction model proportional to velocity:

$$\frac{dv_x}{dt} = -\frac{5}{7}\rho_g g \frac{v_x}{\sqrt{v_x^2 + v_y^2 + v_z^2}}, \quad \frac{dv_z}{dt} = -\frac{5}{7}\rho_g g \frac{v_z}{\sqrt{v_x^2 + v_y^2 + v_z^2}}$$

This document and the information contained herein are the property of Matthieu PHILIPPE.
They must not be copied or disclosed to third parties without prior written authorization.

- The derivative of v_y accounts for the terrain slope via the `dAltiSol` function:

$$\frac{dv_y}{dt} = -\frac{5}{7}\rho_g g \frac{\text{dAltiSol}(v_x, v_z)}{\sqrt{v_x^2 + v_y^2 + v_z^2}}$$

- The derivatives of the spin components are assumed to be zero (no spin decay during rolling for simplification).

Note: The simplifications in the rolling model are made to ensure rapid computation, as the model is designed to process real-time radar data of the ball's initial flight, which will be discussed in the next chapter.

—

3.2.3 The `AltiSol` and `dAltiSol` Functions

The `AltiSol` and `dAltiSol` functions are used to model the terrain altitude and its slope, respectively.

3.2.3.1 The `AltiSol` Function

- **Role**: Computes the terrain altitude at a position (x, z).
- **Inputs**:
  - `Longueur`: Position x (m).
  - `largeur`: Position z (m).
- **Output**:
  - `Altitude`: Terrain altitude at position (x, z) (m).
- **Logic**:
  - If $x < 30$ m, the altitude is set to 0 (flat terrain).
  - Otherwise, the altitude is also set to 0, but a commented line suggests a potential model for sloped terrain.

3.2.3.2 The `dAltiSol` Function

- **Role**: Computes the derivative of the terrain altitude with respect to the horizontal position, used to model the slope during rolling.
- **Inputs**:
  - `vLongueur`: Velocity v_x (m/s).
  - `vlargeur`: Velocity v_z (m/s).
- **Output**:
  - `vAltitude`: Derivative of the terrain altitude.
- **Logic**:

This document and the information contained herein are the property of Matthieu PHILIPPE.
They must not be copied or disclosed to third parties without prior written authorization.

- If $v_x < 30\,\text{m/s}$, the derivative is set to 0 (flat terrain).
- Otherwise, the derivative is also set to 0, but a commented line suggests a potential model for sloped terrain.

—

3.2.4 Event Detection Functions

The `events` and `events2` functions are used to detect discrete events during the simulation, such as the ball's impact with the ground or its stopping condition.

3.2.4.1 The `events` Function

- **Role**: Detects when the ball hits the ground ($y \leq \text{Altitude}(x, z)$).
- **Inputs**:
  - `t`: Current time (s).
  - `x`: State vector.
- **Output**:
  - `value`: Value used by the ODE solver to detect the event.
- **Logic**:
  - If the ball's height `x(3)` is less than the terrain altitude at position (`x(1)`, `x(5)`), the event is triggered (`value = 0`).
  - Otherwise, the function returns the difference between the ball's height and the terrain altitude.

3.2.4.2 The `events2` Function

- **Role**: Detects when the ball stops rolling ($v_x \leq 0$).
- **Inputs**:
  - `t`: Current time (s).
  - `x`: State vector.
- **Output**:
  - `value`: Value used by the ODE solver to detect the event.
- **Logic**:
  - The event is triggered when the horizontal velocity `x(2)` is less than or equal to 0.

—

3.2.5 Numerical Integration with `ode`

The numerical solution of the differential equations is performed using Scilab's `ode` function. This function uses the Runge-Kutta 4th order method (RK4) by default to integrate the ODE system.

This document and the information contained herein are the property of Matthieu PHILIPPE.
They must not be copied or disclosed to third parties without prior written authorization.

3.2.5.1 Calling the ode Function

In the main Golfball function, the ode function is called as follows:

```
[Pos, rd] = ode('roots', Init0, T0, 0:0.1:Tfin, flightg, 1, events);
```

Listing 3.1: Calling the ode function for the flight phase

- **Parameters**:
  - 'roots': Integration method (RK4 by default).
  - Init0: Initial state vector.
  - T0: Initial time (usually 0).
  - 0:0.1:Tfin: Time grid with a step size of 0.1 s.
  - flightg: Function defining the ODEs for the flight phase.
  - 1: Indicates that events should be detected.
  - events: Event detection function for impacts.
- **Outputs**:
  - Pos: Matrix containing the state vector at each time step.
  - rd: Vector containing the times at which events (e.g., impacts) occur.

Note: The fixed time step of 0.1 s is chosen to balance computational efficiency and accuracy, as the model is designed to process real-time radar data of the ball's initial flight, which will be discussed in the next chapter.

This document and the information contained herein are the property of Matthieu PHILIPPE.
They must not be copied or disclosed to third parties without prior written authorization.

Chapter 4

Hardware



Figure 4.1: The suitecase

This document and the information contained herein are the property of Matthieu PHILIPPE.
They must not be copied or disclosed to third parties without prior written authorization.

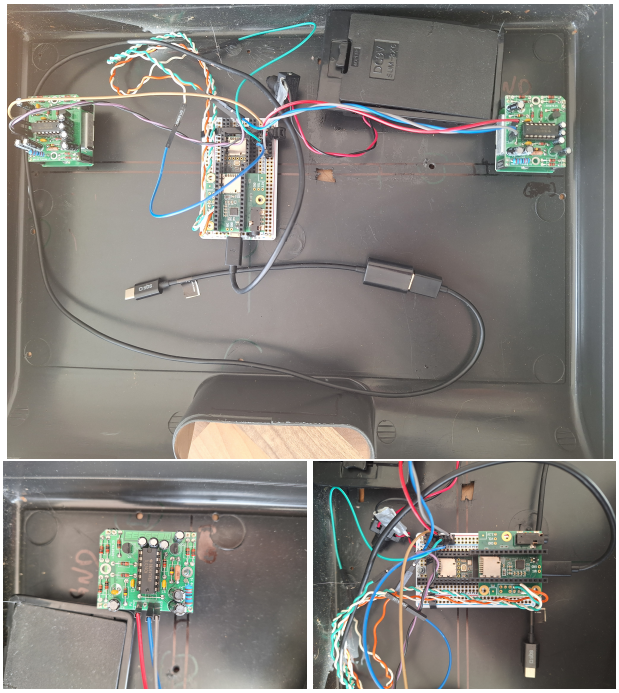


Figure 4.2: Internal with Arduino, Radar, Amplifier, Audio Boards



Figure 4.3: Tiny Arduino

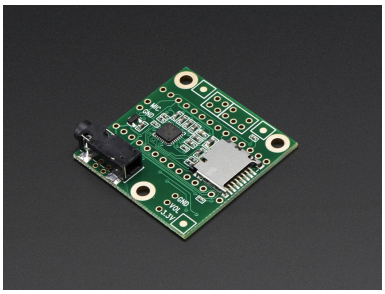


Figure 4.4: Audio Board

This document and the information contained herein are the property of Matthieu PHILIPPE.
They must not be copied or disclosed to third parties without prior written authorization.

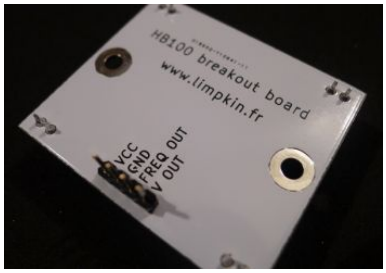


Figure 4.5: Audio Amplifier Board



Figure 4.6: Radar Board

This document and the information contained herein are the property of Matthieu PHILIPPE.
They must not be copied or disclosed to third parties without prior written authorization.

Chapter 5

Data Anlysis

5.1 Filtered Radar Captures

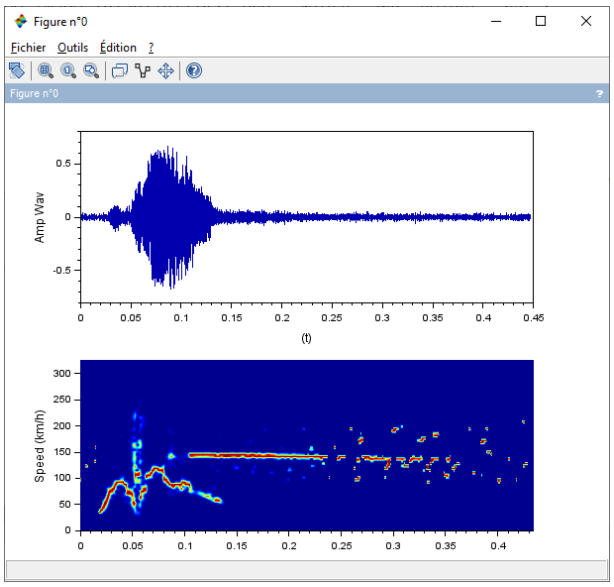


Figure 5.1: Wav Radar Captures and its echo

This document and the information contained herein are the property of Matthieu PHILIPPE.
They must not be copied or disclosed to third parties without prior written authorization.

5.1.1 Filtering

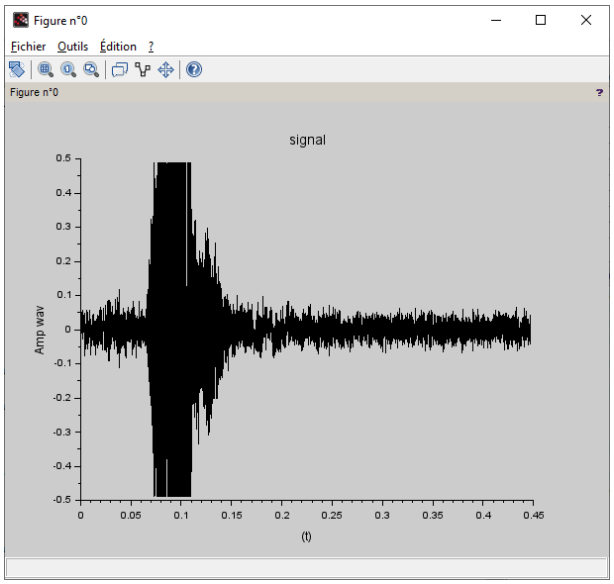


Figure 5.2: Captured Raw Data Wav

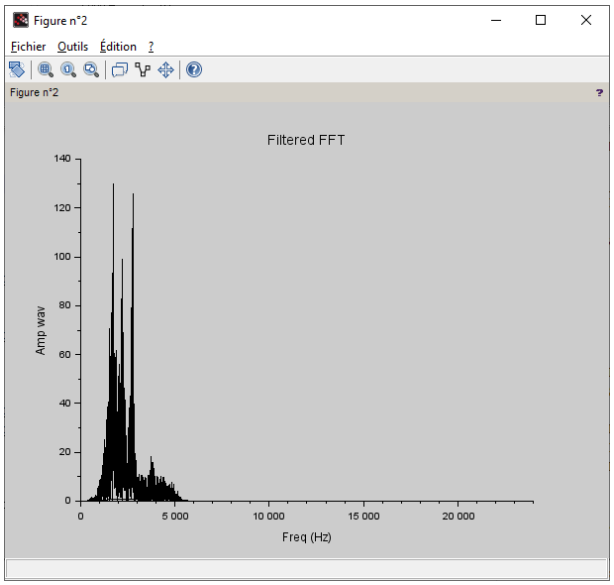


Figure 5.3: FFT of Filtered Raw Data

This document and the information contained herein are the property of Matthieu PHILIPPE.
They must not be copied or disclosed to third parties without prior written authorization.

5.1.2 Final Result

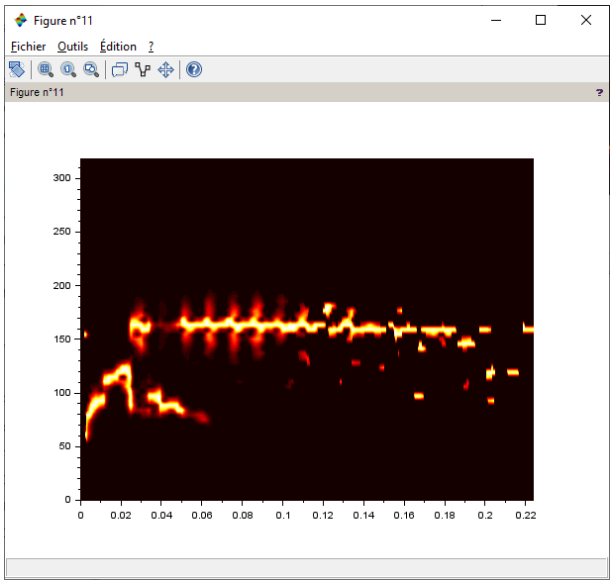


Figure 5.4: My power density Algorithm Results to identify Speed and BackSpin

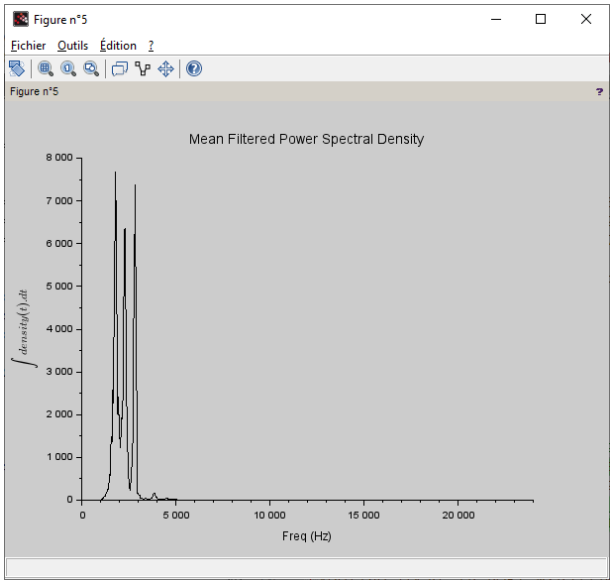


Figure 5.5: Results of normalized mean filtered power spectral density

This document and the information contained herein are the property of Matthieu PHILIPPE.
They must not be copied or disclosed to third parties without prior written authorization.

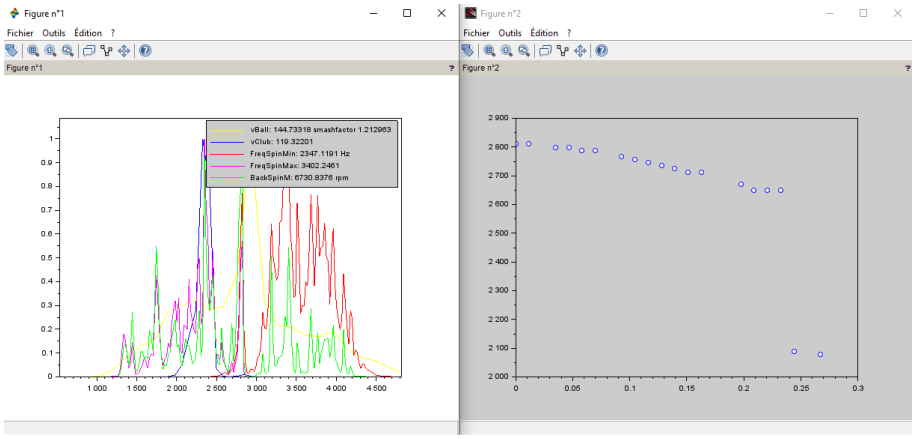


Figure 5.6: Results of the filtered Density and caracteristics identification

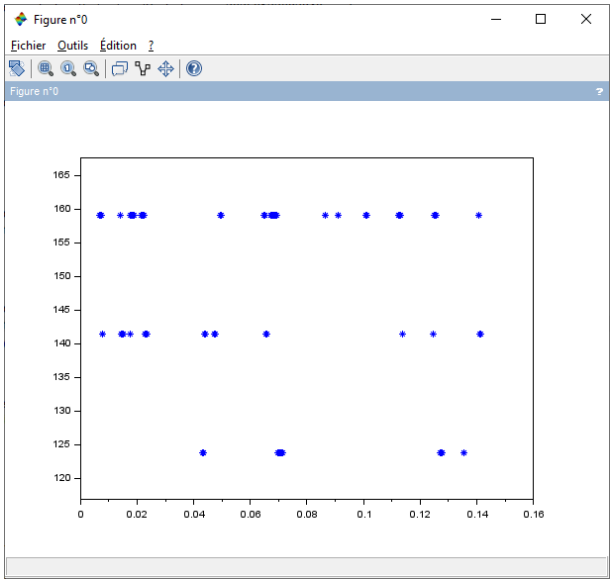


Figure 5.7: Speed and Backspin echos

This document and the information contained herein are the property of Matthieu PHILIPPE.
They must not be copied or disclosed to third parties without prior written authorization.

Chapter 6

Radar Doppler Processing Pipeline: from Scilab to Python

This chapter documents the complete signal-processing chain used to extract ball speed, spin total and launch azimuth from the two 10.525 GHz Doppler radars (audio acquisition at $F_s = 44100$ Hz, one channel per radar), the corrections applied to the historical Scilab code (`sci/FFT_Mat.sce`, commit `73ef2d8`), and the Python port (`sci/pipe_validation.py`) used for blind validation on the 2019-02-24 F7 captures (9 shots) plus 2 generated model signals.

6.1 Two-radar geometry and the Doppler observables

Each radar measures only the projection of the ball velocity on its line of sight:

$$f_i = \frac{2}{\lambda}\,\mathbf{V}\cdot\mathbf{u}_i = K_V\,V_{r,i}, \qquad K_V \approx 19.49\ \text{Hz/(km/h)}, \quad \lambda = c/f_0 \approx 0.02848\ \text{m} \tag{6.1}$$

With radars at $R_1 = (0, -b/2)$ and $R_2 = (0, +b/2)$ in the horizontal plane (axis x = nominal shot direction, z = lateral, $b = 0.30$ m), ball at $P = (x, z)$, $L_1 \approx L_2 \approx L$:

$$f_1 = K\,\frac{V_x x + V_z(z + b/2)}{L}, \qquad f_2 = K\,\frac{V_x x + V_z(z - b/2)}{L} \tag{6.2}$$

Summing cancels the $\pm b/2$ terms (common mode), subtracting cancels the longitudinal terms (differential mode):

$$f_C = \frac{f_1 + f_2}{2} \approx K\,V_x \quad \text{“how fast it goes forward”} \tag{6.3}$$

$$f_\Delta = \frac{f_1 - f_2}{2} = K\,\frac{V_z\,b}{2L} \quad \text{“how fast it drifts laterally, seen through the parallax } b/L\text{”} \tag{6.4}$$

The ratio gives back the slope of the velocity vector, i.e. the tangent of the horizontal launch angle α (angle between $\mathbf{V}$ and the x axis in the x–z plane):

This document and the information contained herein are the property of Matthieu PHILIPPE.
They must not be copied or disclosed to third parties without prior written authorization.

$$\tan\alpha = \frac{V_z}{V_x} = \frac{2L}{b}\,\frac{f_\Delta}{f_C} \quad\Longrightarrow\quad \alpha(t) = \arctan\!\left[\frac{2L(t)}{b}\,\frac{f_\Delta(t)}{f_C(t)}\right], \qquad L(t) = L_0 + \int_{t_0}^{t} V_x(\tau)\,d\tau \tag{6.5}$$

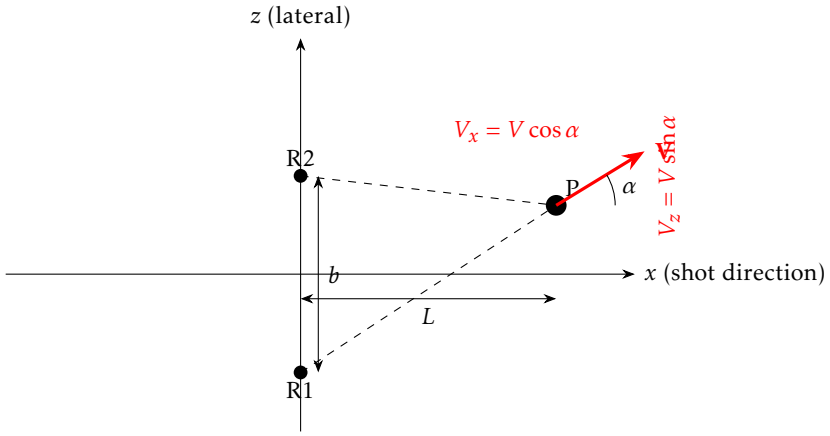


Figure 6.1: Top view of the two-radar setup. f_C carries V_x (speed), f_Δ carries V_z (azimuth α in the x–z plane). The factor $2L/b$ corrects the parallax: the farther the ball, the smaller the frequency difference for the same V_z.

6.2 STFT and ridge tracking

For each channel, a short-time Fourier transform gives the spectrogram $P_i(k,m) = |X_i(k,m)|^2$:

- Scilab `animDensite` (`FFT_Mat.sce`): Hann window 512, hop 16, threshold in dB per frame, normalized by the frame max;
- Python `spectrogram` (`pipe_validation.py`): Hann window $N = 2048$ (46 ms), hop 1024 (50% overlap), parabolic sub-bin interpolation of the peak.

The ridge frequency of frame m is the peak bin refined by a parabola:

$$k^* = \arg\max_k P_m(k), \qquad \delta = \frac{0.5\,(P_{k^*-1} - P_{k^*+1})}{P_{k^*-1} - 2P_{k^*} + P_{k^*+1}}, \qquad f_{ridge}(m) = (k^* + \delta)\,\frac{F_s}{N} \tag{6.6}$$

The impact time is the most energetic frame (the metallic club gives a strong, clipped echo):

$$t_0 = \arg\max_m \sum_k P_m(k) \tag{6.7}$$

6.3 Speed: median of the post-impact ridge vs global max

The global max of the time-summed spectrum, $\hat{f}_B = f_5[\arg\max_k \sum_m M_5(m,k)^2]$ (Scilab `Info`), is dominated by the strong clipped club echo (6.6–14.8% of samples at ±0.489), whose spectrum is broad and flat. The ball produces a narrow ridge that *lasts* the whole flight. Hence the Python estimator:

$$\hat{V} = \frac{1}{19.49}\,\underset{m\in[t_0,\,t_0+50\,\mathrm{ms}]}{\mathrm{median}}\, f_{ridge}(m) \tag{6.8}$$

This document and the information contained herein are the property of Matthieu PHILIPPE.
They must not be copied or disclosed to third parties without prior written authorization.

Each frame votes one frequency, unweighted: the median tolerates up to 50% of aberrant frames (club residue, false lobes), whereas the energy-weighted sum lets one strong false frame pull the result. Measured on the 11 CSV: MAE 5.7% radial, 1.9% after the $\cos 20°$ geometric correction. On a microcontroller (Arduino), the impact detection is $O(1)$ per frame and a max over the post-impact window is acceptable when t_0 is reliable; the median is the safety margin when it is not.

6.4 Spin: patch periodicity vs spectral excursion

The ball is partially covered with aluminium tape (one dominant scatterer, "the patch"). Two estimators of the *total* spin exist:

1. **Spectral excursion** (Scilab `GetSwing/Info2`): the surface point speed is $v_t = r\omega$, so the Doppler spread is

$$\Delta f_{spin} = \frac{f_+ - f_-}{2} = \frac{2r\omega}{\lambda}, \qquad SpinTotal_{RPM} = \frac{\Delta f_{spin}}{K_S}, \quad K_S \approx 0.1567 \text{ Hz/RPM} \tag{6.9}$$

   This uses the two *extreme edges* of the micro-Doppler spectrum – the weakest points, hence the most buried in noise and ADC clipping. Measured MAE: 41.4% (`GetSwing`) to 51–68% (variants).

2. **Patch periodicity** (Python): the patch produces one radar "flash" per revolution, whatever the tilt β of the spin axis, so the Hilbert envelope of the post-impact signal contains a line at

$$f_{rot} = \frac{SpinTotal}{60} \quad \Longrightarrow \quad \boxed{SpinTotal_{period} = 60\, f_{rot}} \tag{6.10}$$

   f_{rot} is the envelope-spectrum peak with the best harmonic score $\sum_q P(k_q)$ for $f_q \approx k f_{rot}$ (avoids sub-harmonics and the degenerate 2-flashes-per-turn case when the spin axis is aligned with the line of sight). Measured MAE: **19.9%** on the 9 real shots, and **−0.1%** (3996 vs 4000 RPM) on the clean generated model signal.

Both estimators measure the *total* spin $\omega = \sqrt{\omega_{back}^2 + \omega_{side}^2}$, not the backspin: the flash frequency is $\omega/2\pi$ whatever β, and the excursion is $2r\omega/\lambda$. Splitting BackSpin/SideSpin requires β, identified elsewhere (curvature of $\alpha(t)$, or the `SpinAxis` of `GetSwing`: $\beta \approx \text{asin}[Spin_3/\sqrt{((Spin_1 + Spin_2)/2)^2 + Spin_3^2}]$, qualitative only: MAE $\approx$ 15–18°).

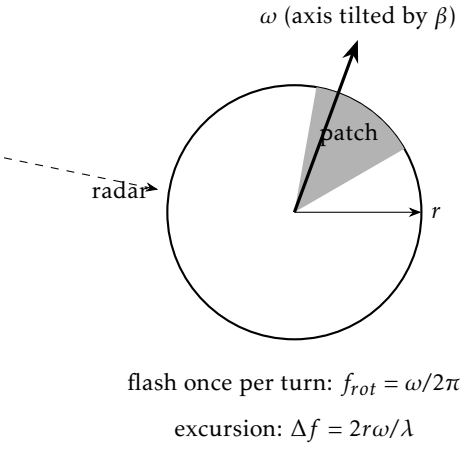


Figure 6.2: The aluminium patch as a single dominant scatterer. Both the flash periodicity and the spectral excursion measure the *total* spin ω; neither separates backspin from sidespin.

This document and the information contained herein are the property of Matthieu PHILIPPE.
They must not be copied or disclosed to third parties without prior written authorization.

6.5 Stereo pairing of the two ridges

Tracking the FFT maximum of each radar independently can pick two different micro-Doppler lobes and produce a false azimuth (errors > 20° observed). The Python tracker pairs them: knowing f_1 (ridge of R1), f_2 is searched in the R2 spectrum within ±300 Hz of f_1, with the coherence constraint $|f_1 - f_2| \lesssim 30$ Hz (geometry, $b = 0.30$ m). Then $\alpha(t)$ follows from eq. above, with

$$\alpha_0 = \underset{t\in[t_0,t_0+50\,\mathrm{ms}]}{\mathrm{median}}\ \alpha(t), \qquad \Delta\alpha_{BF} = \underset{W_F}{\mathrm{median}}(\alpha) - \underset{W_0}{\mathrm{median}}(\alpha) \tag{6.11}$$

Measured: α_0 resolved 9/9 (−3.1° to −0.2°); sign of $\Delta\alpha$ vs reference sidespin 0/9 because the reference files encode |sidespin| only (signed left/right dataset is the next priority).

6.6 Quality controls ("non identifiable" is a valid output)

- symmetry of the micro-Doppler edges: $|(f_C - f_-) - (f_+ - f_C)| < \eta\,\Delta f_{spin}$, $\eta = 0.2$;
- inter-radar coherence: $|Spin_1 - Spin_2|/\mathrm{mean} < 20\%$;
- edge/period concordance < 20% to merge the two spin estimators, else periodicity alone;
- inter-radar speed divergence > 15% ⇒ reject the false-lobe channel;
- validation targets: $\mathrm{MAE}_V < 2$ km/h, $\mathrm{MAE}_{spin} < 500$ RPM (8%), $\mathrm{MAE}_{\alpha_0} < 1°$, > 90% correct curvature signs, $\mathrm{MAE}_\beta < 7°$ then 3–5°.

6.7 Scilab → Python: corrections and validation results

Corrections committed to `sci/FFT_Mat.sce` (commit `73ef2d8`, merged to `main`):

- `deltaF`: added the missing 3.6 factor (km/h → m/s) and harmonized $r = 0.04267/2$, now consistent with `Freq2RpmSpin`;
- `animDensite2/3`: keep the *low* half of the spectrum ($M1(:, 1:\$/2)$) matching the frequency vector (was `$/2:$-1`, high half, broken mapping); threshold set to 0 instead of −1000 (which crushed the peak after normalization);
- `Enveloppe`: I/Q demodulation at the band-center carrier $f_c = (f_b + f_h)/2$ instead of $\cos(t \cdot F_s) = \cos(n)$ (demodulation at $F_s/2\pi \approx 7$ kHz, out of band).

Blind validation on the 11 CSV files (references encoded in the file names), port of the exact Scilab pipes (README "Simples Scilab Commands" + `testsFFT.sci` line 1029) vs the Python pipe:

Table 6.1: Measured results, 11 CSV (9 real F7 shots + 2 model signals).

Estimator	MAE V_{ball} radial	MAE corr. cos 20°	MAE SpinTotal
Scilab `GetSwing` (port)	5.7%	≈1.9%	41.4% (excursion)
Scilab robust variants	5.7%	≈1.9%	60–68%
Python `pipe_validation.py`	**5.7%**	**1.9%**	**19.9%** (periodicity)

Conclusion: the Scilab chain is *good for speed and impact* (identical MAE to Python) and indicative for spin; the Python pipe adds the patch-periodicity spin estimator, the stereo pairing for α, and the quality controls – the gains come from choosing the right observable, not from heavier math.

This document and the information contained herein are the property of Matthieu PHILIPPE.
They must not be copied or disclosed to third parties without prior written authorization.

6.8 Processing pipeline

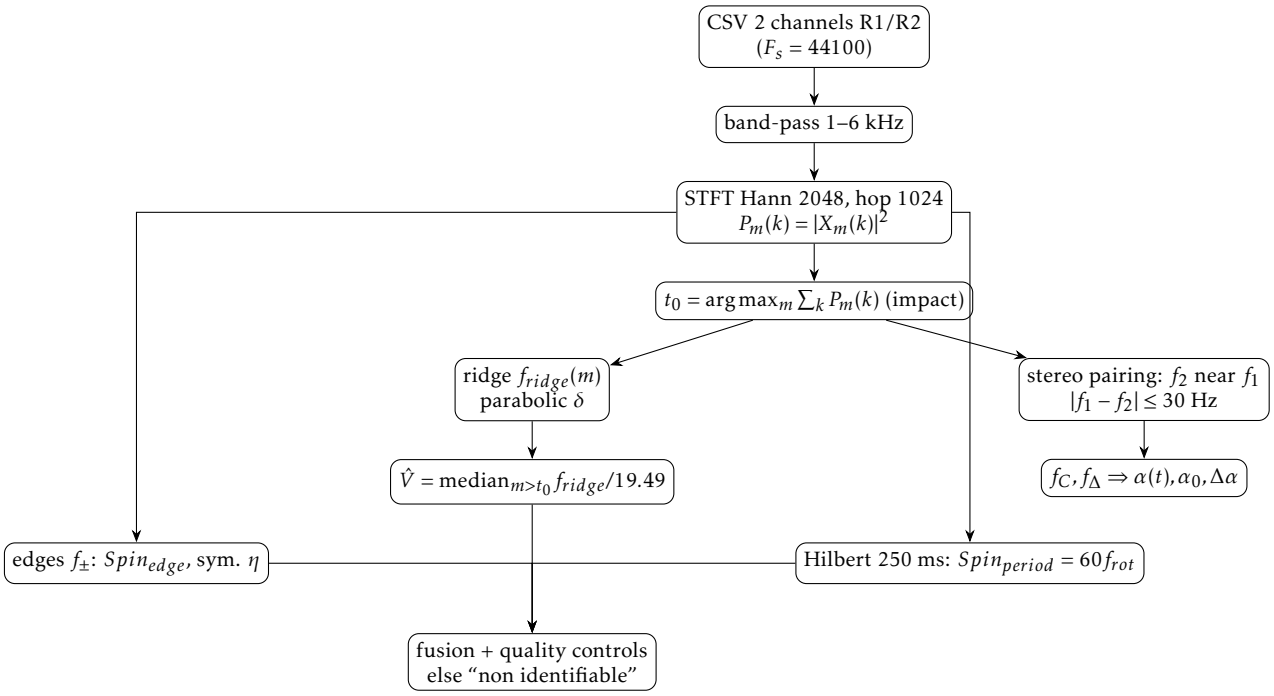


Figure 6.3: Python validation pipeline (`sci/pipe_validation.py`). Scilab equivalents: `animDensite` (STFT), `Info/GetSwing` (speed), `animHilbert` (envelope), `acos(f2/f1)` (rudimentary azimuth in `testsFFT.sci`).

6.9 Spectrograms: measured captures vs theoretical model

The following figures show the measured spectrograms of the 2019-02-24 F7 captures (R1/R2, vertical axis in km/h via $f/19.49$, ball ridge in cyan, impact marked), and the *generated* theoretical captures (`gen_capture_modele.py`: Doppler carrier $f_D(t) = 19.49\,V(t)\cos\alpha_{radar}$ + patch phase modulation $e^{-i(2r/\lambda)\sin\omega t}$ + club peak at $t_0 = 100$ ms + 28 dB noise). The model captures are references of what the processing *should* obtain on a clean signal – on the 102 m model shot the pipe recovers the spin to −0.1 %.

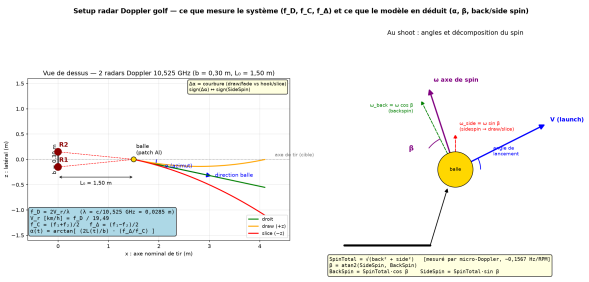


Figure 6.4: Radar setup geometry (also in README).

This document and the information contained herein are the property of Matthieu PHILIPPE.
They must not be copied or disclosed to third parties without prior written authorization.

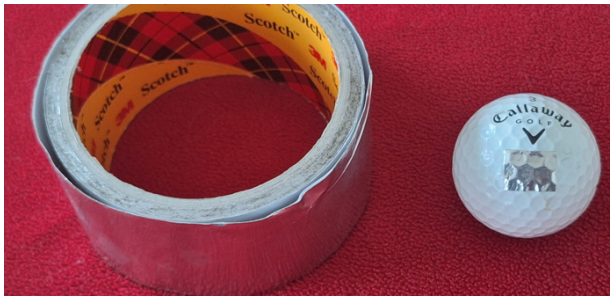


Figure 6.5: Patch on the ball (aluminium tape).

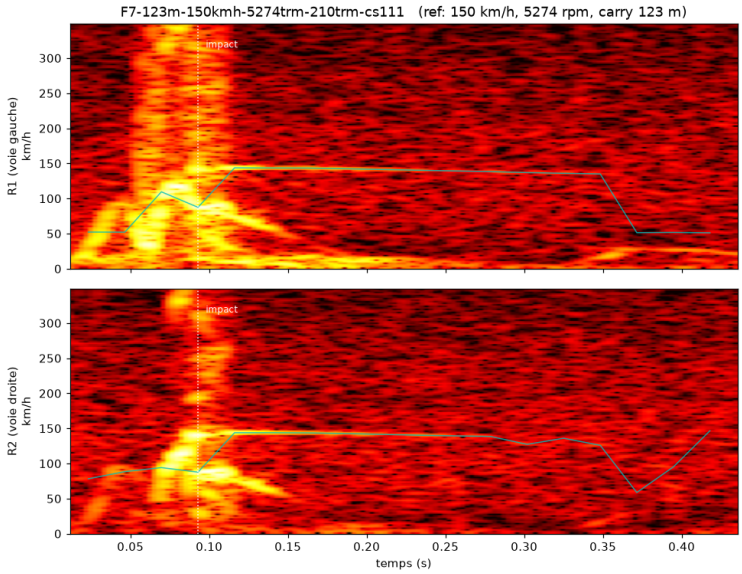


Figure 6.6: Measured spectrogram, shot 123 m / 150 km/h / 5274 RPM (SkyTrak reference).

This document and the information contained herein are the property of Matthieu PHILIPPE.
They must not be copied or disclosed to third parties without prior written authorization.

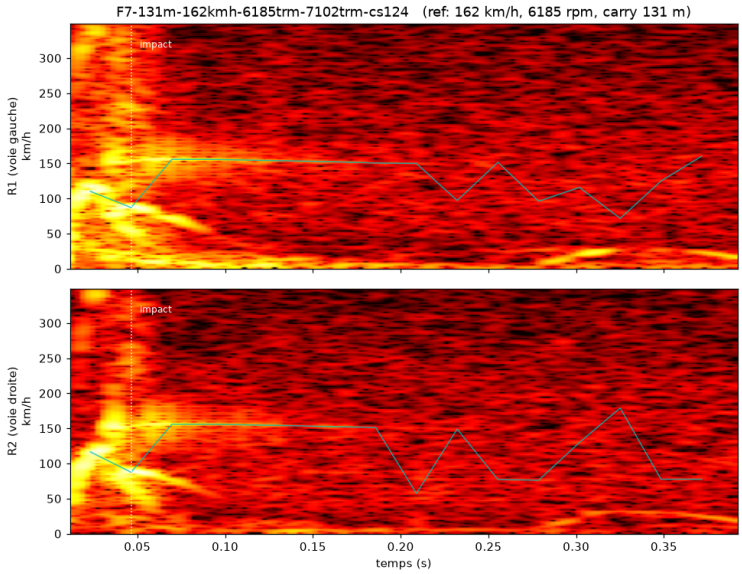


Figure 6.7: Measured spectrogram, shot 131 m / 162 km/h / 6185 back + 7102 side RPM ($\beta \approx 48.9°$, strong curvature).

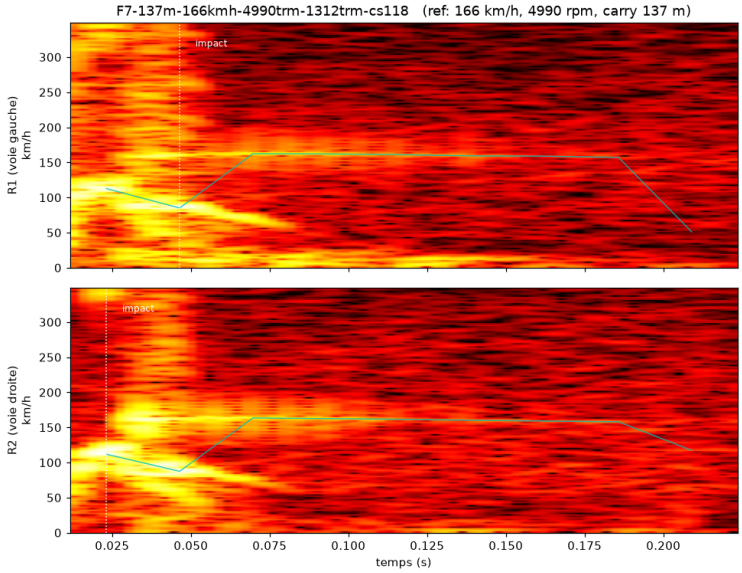


Figure 6.8: Measured spectrogram, shot 137 m / 166 km/h / 4990+1312 RPM ($\beta \approx 14.7°$).

This document and the information contained herein are the property of Matthieu PHILIPPE.
They must not be copied or disclosed to third parties without prior written authorization.

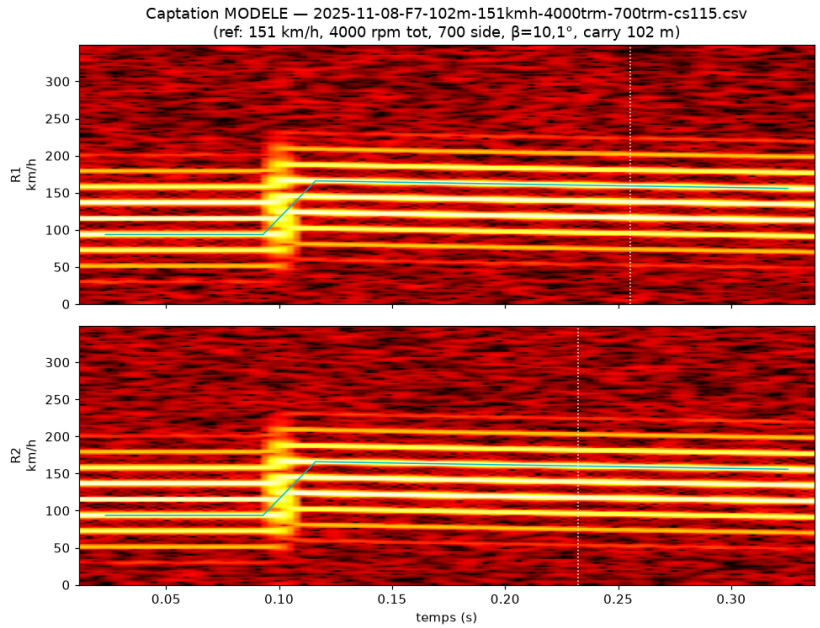


Figure 6.9: Theoretical (generated) capture, config F7 151 km/h, launch 16.5°, spin 4000 RPM, side 700 RPM ($\beta = 10.1°$), club 115 km/h, carry 102 m.

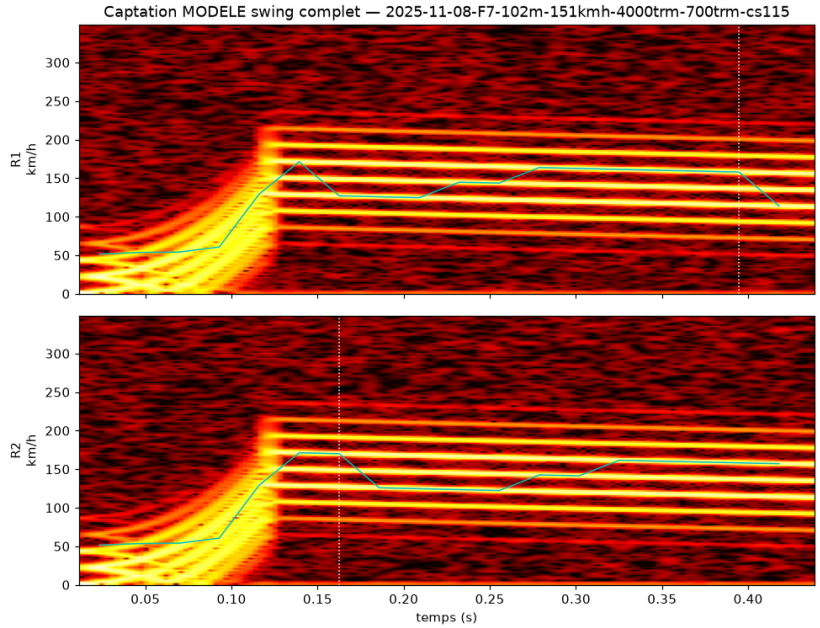


Figure 6.10: Generated capture, same config, swing-complete 450 ms version (v2).

This document and the information contained herein are the property of Matthieu PHILIPPE.
They must not be copied or disclosed to third parties without prior written authorization.

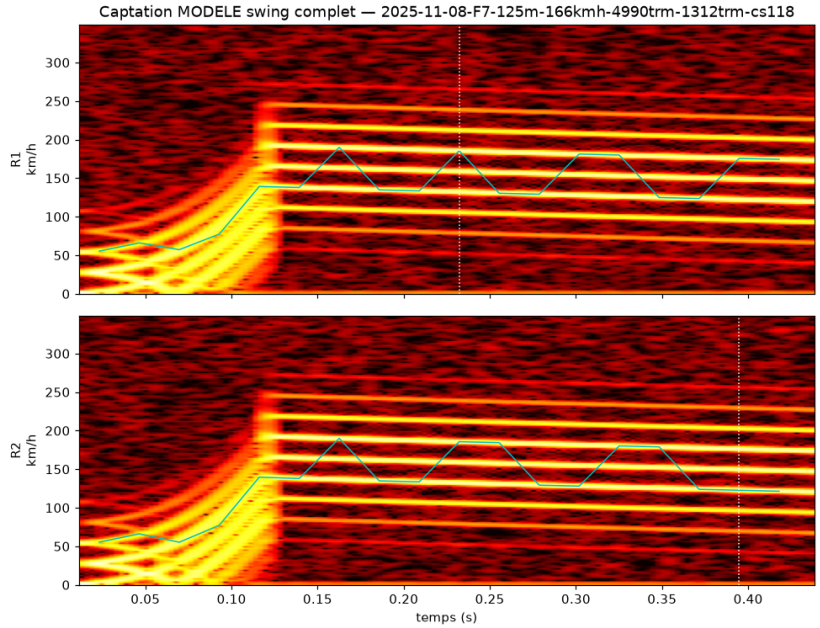


Figure 6.11: Generated capture with the real 137 m shot parameters (166 km/h, 4990/1312 RPM, club 118).

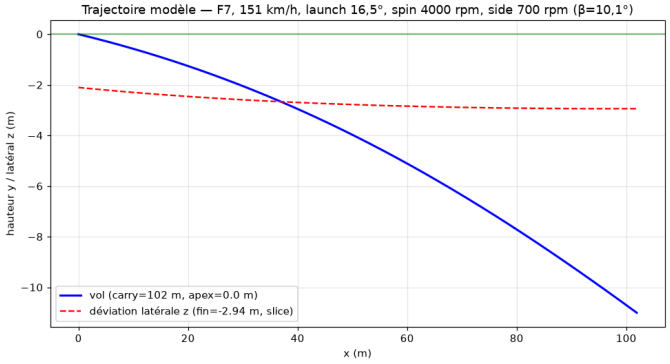


Figure 6.12: Model trajectory (flight + lateral deviation of the slice) for the 2025-11-08 generated capture.

This document and the information contained herein are the property of Matthieu PHILIPPE.
They must not be copied or disclosed to third parties without prior written authorization.

Chapter 7

Listings

7.1 Golfball.sci

```
// Copyright (C) 2016 - Corporation - Author
//
//
// Date of creation: 16 juin 2016
//
//SCI2C: DEFAULT_PRECISION= DOUBLE
//
// ReadWave : FFT_Mat.sce
//
```

Listing 7.1: Listing of the golf ball fligth

//

This document and the information contained herein are the property of Matthieu PHILIPPE.
They must not be copied or disclosed to third parties without prior written authorization.

Conflicts of Interest

The authors declare no conflict of interest.

This document and the information contained herein are the property of Matthieu PHILIPPE.
They must not be copied or disclosed to third parties without prior written authorization.

Bibliography

[Ala98] Leroy Alaways. Aerodynamics of the curve-ball: An investigation of the effects of angular velocity on baseball trajectories. 01 1998.

[ASC+11] Firoz Alam, Tom Steiner, Harun Chowdhury, Hazim Moria, Iftekhar Khan, Fayez Aldawi, and Aleksandar Subic. A study of golf ball aerodynamic drag. *Procedia Engineering*, 13:226–231, 12 2011.

[BR] Burglund Brett and Street Ryan. Golf ball flight dynamics 5-13-2011 1. https://www.math.union.edu/ wangj/courses/previous/math238w13/Golf

[GAS] Gilbert GASTEBOIS. L'effet magnus. https://ggastebois.fr/java/magnus/Magnus_theorie.pdf.

[LJ94] B. B. Lieberman and S. H. Johnson. An analytical model for ball-barrier impact. *Proceedings of the 1994 World Scientific Congress of Golf*, edited by A. J. Cochran and M. R. Farrally E and FN Spon, London:375–380, July 1994.

[Mag53] HG Magnus. Über die abweichung der geschosse—about the deviation of the projectiles. *Poggendorf's Ann. Phys. Chem.*, 88:1–28, 1853.

[PAL05] GRANT PALMER. *Physics for Game Programmers*. APRESS, 2005.

[Pen01] Albert Penner. The physics of golf: The optimum loft of a driver. *American Journal of Physics - AMER J PHYS*, 69, 05 2001.

[Pen02a] Albert Penner. The physics of putting. *Canadian Journal of Physics*, 80:83–96, 02 2002.

[Pen02b] Albert Penner. The run of a golf ball. *Canadian Journal of Physics*, 80:931–940, 07 2002.

[PWB76] J K HARVEY P W BEARMAN. Flight : Golf ball aerodynamics. In Imperial College Of Science and Technolgy, editors, *Journal of The AERONAUTICAL QUARTERLY*, volume v27, pages 112–122, 1976.

[RL10] Woo-Jin Roh and Chong-Won Lee. Golf ball landing, bounce and roll on turf. *Procedia Engineering*, 2:3237–3242, 06 2010.

[SS94] Alexander Smits and D.R. Smith. A new aerodynamic model of a golf ball in flight. *Science and Golf II*, pages 340–347, 01 1994.

This document and the information contained herein are the property of Matthieu PHILIPPE.
They must not be copied or disclosed to third parties without prior written authorization.